Subcellular proteomic analysis of the insula in alcohol dependence

NBIS support

Yuan Li

Table of contents

1 Support Request **2**

2 Data **2**

3 Import and process the data **3**

4 Normalization **12**

5 QC **21**
5.1 PCA 21
5.2 Outlier 23

6 Statistical test **25**
6.1 PCA on DEGs 32
6.2 Barplot of significant proteins abundance (adj.P.Val<0.2) 33
6.3 How many proteins are shared: upset plot 34
6.4 How many shared: statistics 35

7 Functional pathway analysis **36**
7.1 Over representation analysis 36
7.1.1 Dotplot showing significant KEGG pathways 39
7.1.2 Check the raw count 40
7.1.3 Look into genes associated with pathways of interest 43

8 Pathway analysis on all three compartments together **53**

9 Drug target analysis **54**

10 Deliverables **58**

11 Directory **58**
11.1 Practical info 58
11.2 Software 58

12 Data responsibility **58**

13 Acknowledgments **58**

NBIS ID: 7314
Report Version: 1.0
Request by: Philippe Melas (philippe.melas@ki.se)
Principal Investigator: Philippe Melas (philippe.melas@ki.se)
Organisation: KI
NBIS Staff: Yuan Li (yuan.li@nbis.se) **conda env:** TTN

1 Support Request

7314 In brief, the data that we have, derive from the following experimental design: Brain samples from alcohol dependent patients (N=12 cases) and matched controls (N=12 controls) were first submitted to subcellular fractionations, which yielded three fractions (i.e., synaptosomes, cytosol, insoluble fraction). All three fractions (total N=72 samples) were subjected to TMT-based quantitative proteomics. Data were provided by a commercial company (Creative Proteomics) in .xlsx format. The company also performed and provided results from basic bioinformatic comparisons (inc. differential expression, pathway analysis, etc.), which revealed some statistical significances between cases and controls on the nominal p-level. We would now like to re-analyze the data and prepare material, e.g., Figures/Tables and Methods, for a publication.

2 Data

The data has already been filtered by the company (e.g. removed proteins with CONTA, REV… as prefix of their names, some **potential** contaminating proteins etc. were kept for they may also represent real proteins)

Because there is a big batch effect (between G1 and G2, see the PCA plot in ../results/old/totalAbundanceNormalized/PELLET/differentialAnalysis/Comparisons_loess0.1/Norm-stats-report-Comparisons_loess0.1.pdf), and there are almost no significant protein. So I decided to renormalize the raw intensity, and see how it will affect the results.

3 Import and process the data

```r
suppressPackageStartupMessages({
  library(readxl)
  library(purrr)
  library(EDASeq)
  library(limma)
  library(emmeans)
  library(edgeR)
  library(future.apply)
  library(dplyr)
  library(tidyr)
  library(plyr)
  library(xlsx)
  library(ggplot2)
  library(ComplexHeatmap)
  library(GGally)
  library(org.Hs.eg.db)
  library(AnnotationDbi)
  library(pathview)
  library(UniProt.ws)
  library(GGally)
  library(clusterProfiler)
  library(ggrepel)
  library(enrichR)
})
```

```r
jName = "7314"
inputP = "../data/Creative Proteomics_DATA/"
# dataP = paste0("../data/input_", jName) #
# dir.create(dataP, showWarnings=F, recursive=T) #
designP = paste0("../data/design_", jName) #
dir.create(designP, showWarnings=F, recursive=T) #
outDir = "../results/rerun/"
dir.create(outDir, recursive = T, showWarnings = F)
```

```r
print("Make the design table without ref...")
print("We will get meta data info from sheet 5 of each excel file")
print("This is the company normalized data, whose count will NOT be analyzed here...")

print("Get sample names")
file = '~$Data result_CYTOSOLIC extracts (CPJS05122201).xlsx'
```

```r
if (file.exists(file)) {file.remove(file)}
sample_names = list.files(inputP, pattern=").xlsx$", recursive = T, full.names = F) %>%
  sapply(., function(i) strsplit(i, " ")[[1]][1]) %>%
  as.character()


print("Import all three datasets together...")
file = '~$Data result_CYTOSOLIC extracts (CPJS05122201).xlsx'
if (file.exists(file)) {file.remove(file)}
inputfiles = list.files(inputP, pattern=").xlsx$", recursive = T, full.names = T)
dataTable.list0 = suppressWarnings(map(inputfiles, function(x) {
  dat = read_excel(x, sheet=5, col_names=TRUE, na="", skip=0)
  ncol = ncol(dat)
  dat = read_excel(x, sheet=5, col_names=TRUE, na="", skip=0,
        col_types=c(rep("text", 4), rep("numeric", ncol(dat)-4-1),
                    rep("text", 1)))
  dat = dat %>%
    .[, c('Protein IDs', "Gene names",
          grep('intensity',
               colnames(dat), value=T))]
  print("Only Pellet has two pool samples for each batch, will remove them...")
  dat = dat %>% .[, colnames(dat)[!grepl('MIX', colnames(dat))]]
  colnames(dat) = gsub(" ", "_", colnames(dat))
  dat
  })) %>% setNames(sample_names)


design.table.list = suppressWarnings(map(seq_along(dataTable.list0), function(i) {
  dataTable = dataTable.list0[[i]]
  sampleNames = colnames(dataTable)[3:ncol(dataTable)]
  tab = sapply(sampleNames, function(j) {
    strsplit(j, "_")[[1]][c(3, 4, 5, 6, 7)]
  }) %>% as.character()
  design.table = matrix(tab, ncol=5, byrow=T) %>% as.data.frame() %>%
    na.omit() %>% setNames(c("sample", "Batch", "sampleID", "Fraction", "Condition")) %>%
    dplyr::mutate(sample=paste0(sample, "_", Batch))
  design.table$Condition = tolower(design.table$Condition)
  design.table.filename = paste0(designP, "/design_", sample_names[i], ".tsv")
  write.table(design.table, design.table.filename, quote=F, row.names=F, sep="\t")
  design.table
})) %>% setNames(sample_names)

moreMeta <- tibble::tribble(
  ~sampleID,     ~Sex,    ~Age,  ~RIN,   ~PMI,  ~Condition,   ~pH, ~TissueStorageTime_mont
```

```
  #---------|-----------|------|-------|-------|------------|------|----------------------
   "13005",      "Male",     45,    7.4,   8.48,     "case",   6.2,                   109.4
   "13172",      "Male",     49,    9.4,   7.32,     "case",   6.6,                    85.6
   "13189",    "Female",     59,    7.8,    7.5,     "case",   6.1,                    84.2
   "13193",      "Male",     38,    9.5,   7.78,     "case",   6.7,                    83.5
   "13234",    "Female",     51,    9.6,  10.12,     "case",   6.8,                    78.2
   "13246",      "Male",     59,    8.8,    9.5,     "case",   6.2,                    76.7
   "13271",      "Male",     65,    7.1,  11.97,     "case",   6.3,                    72.4
   "13296",      "Male",     55,    8.9,   7.68,     "case",   6.5,                    69.4
   "13327",      "Male",     58,    8.3,    6.7,     "case",   6.3,                    65.4
   "13346",      "Male",     53,    7.0,    9.3,     "case",   6.2,                    63.3
   "13211",      "Male",     52,    8.8,  12.33,     "case",   6.1,                    81.1
   "13504",      "Male",     50,    7.1,  11.25,     "case",   6.5,                    44.2
   "13350",      "Male",     45,    8.9,   7.08,  "control",   6.7,                    62.7
   "13259",      "Male",     48,    8.5,   8.25,  "control",   6.3,                    74.8
   "13475",    "Female",     58,    7.8,   8.75,  "control",   6.8,                    48.5
   "13414",      "Male",     41,    8.1,   9.85,  "control",   6.7,                    54.5
   "13149",    "Female",     46,    7.5,    7.6,  "control",   6.3,                    88.3
   "13645",      "Male",     58,    9.3,   14.4,  "control",   6.5,                    23.0
   "13428",      "Male",     64,    9.0,   9.22,  "control",   6.7,                    53.5
   "13048",      "Male",     54,    9.3,    8.4,  "control",   6.8,                   103.2
   "13170",      "Male",     59,    8.9,  11.65,  "control",   6.7,                    86.0
   "13032",      "Male",     52,    8.1,   8.97,  "control",   6.4,                   105.9
   "13068",      "Male",     48,    9.5,  12.78,  "control",   6.8,                   100.1
   "13298",      "Male",     50,    8.5,  10.67,  "control",   6.7,                    69.3
 )

 design.table.list = lapply(design.table.list, function(t) {
   dplyr::full_join(t, moreMeta)
 })
```

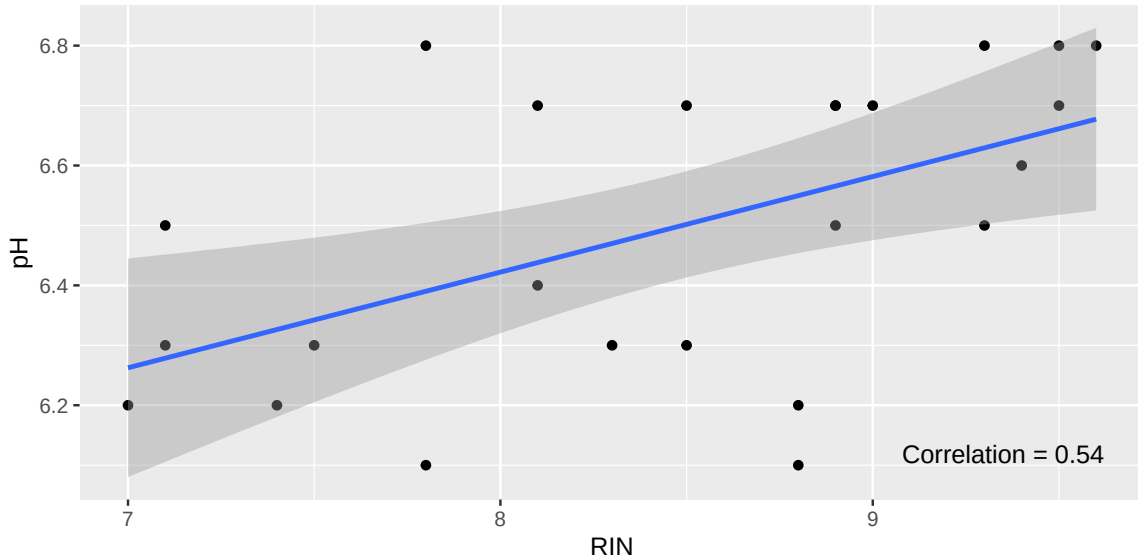
```
[1] "Make the design table without ref..."
[1] "We will get meta data info from sheet 5 of each excel file"
[1] "This is the company normalized data, whose count will NOT be analyzed here..."
[1] "Get sample names"
[1] "Import all three datasets together..."
[1] "Only Pellet has two pool samples for each batch, will remove them..."
[1] "Only Pellet has two pool samples for each batch, will remove them..."
[1] "Only Pellet has two pool samples for each batch, will remove them..."
```

```
print("Get sample names")
sample_names = list.files(inputP, pattern=").xlsx$", recursive = T, full.names = F) %>%
  sapply(., function(i) strsplit(i, " ")[[1]][1]) %>%
```

```
  as.character()

print("Import all three datasets together")
print("Here we start from the company filtered raw data")
inputfiles = list.files(inputP, pattern=").xlsx$", recursive = T, full.names = T)
dataTable.list1 = suppressWarnings(map(inputfiles, function(x) {
  get_dat = function(j) {
    dat = read_excel(x, sheet=j, col_names=TRUE, na="", skip=0)
    ncol = ncol(dat)
    dat = dat %>%
      .[, c('Protein IDs', "Gene names",
            grep('corrected',
                 colnames(dat), value=T))]
    colnames(dat) = gsub("Reporter intensity corrected ", "", colnames(dat))
    colnames(dat)[3:ncol(dat)] = paste0(colnames(dat)[3:ncol(dat)], "_G", i)
    print("Those are supposed to be the intensity corrected")
    colnames(dat) = gsub(" ", "_", colnames(dat))
    dat$Protein_IDs = sapply(dat$Protein_IDs, function(i) {
      strsplit(i, ";")[[1]][1]
    }) %>% as.character()
    return(dat)
  }
  i = 1
  j = ifelse(i==1, 1, 3)
  dat1 = get_dat(j)
  i = 2
  j = ifelse(i==1, 1, 3)
  dat2 = get_dat(j)
  dat = merge(dat1,dat2, by="Protein_IDs", all=T)
  dat$Gene_names =
      paste(dat[,"Gene_names.x"], dat[, "Gene_names.y"], sep=";") %>%
        gsub("NA;", "", .) %>% gsub(";NA", "", .) %>% gsub(";NA;", "", .)

  dat = dat %>%
    dplyr::select(Protein_IDs, Gene_names, everything(), -Gene_names.x, -Gene_names.y)
  dat
})) %>% setNames(sample_names)

lapply(1:length(dataTable.list1), function(i) {
  d = dataTable.list1[[i]]
  l = na.omit(d)
  print(paste0("The column names of the dataset ", sample_names[i], " is ", colnames(d)))
  print(paste0("The original number of proteins for the dataset ", sample_names[i], " is '
  paste0("After removing NA-containing proteins, the number of proteins for the dataset ",
```

```
})

print("There are lots of NA-containing proteins, ")
print("if we remove them, we will remove lots of info, so I will keep them")

print("I also notice that compared to the DEP table (sheet5) the company used,")
print("there are extra samples, which represent pool samples, we will use them to calculat
dataTable.list = lapply(dataTable.list1, function(dataTable) {
  samples_newnames = colnames(dataTable)[3:ncol(dataTable)]
    # sapply(colnames(dataTable)[3:ncol(dataTable)], function(s) {
    # paste0(strsplit(s, "_")[[1]][3], "_", strsplit(s, "_")[[1]][4])
    # }) %>% as.character()
  #print(samples_newnames)
  colnames(dataTable) = c('Protein_IDs', "Gene_names", samples_newnames)
  dataTable
})

OUTDIR_QC = paste0(outDir, "/QC/")
dir.create(OUTDIR_QC, recursive = T, showWarnings = F)

pdf(paste0(OUTDIR_QC, "/Barplot_raw_total_intensity.pdf"), h=4, w=10)
sum.list = lapply(names(dataTable.list), function(i) {
  d = dataTable.list[[i]]
  sample.names = grep("_G", colnames(d), value=T)
  total = d[, sample.names] %>% colSums(na.rm=T)
  missingValues = is.na(d[, sample.names]) %>% colSums()
  mat = data.frame(Total_intensity=as.numeric(total),
                   sample=names(total),
                   MissingValues=as.numeric(missingValues))
  mat$sample = factor(mat$sample,
    levels=c(grep("_G1", mat$sample, value=T),
             grep("_G2", mat$sample, value=T)))
  p1=ggplot(mat, aes(x=sample, y=Total_intensity)) +
    geom_bar(stat="identity") +
    theme(axis.text.x = element_text(angle = 45,
                                     vjust = 1, hjust = 1)) +
    ggtitle(paste0(i, "_raw data"))
  print(p1)
  p2=ggplot(mat, aes(x=sample, y=MissingValues)) +
    geom_bar(stat="identity") +
    theme(axis.text.x = element_text(angle = 45,
                                     vjust = 1, hjust = 1)) +
    ggtitle(paste0(i, "_missing values"))
  print(p2)
```

```
  return(mat)
}) %>% setNames(names(dataTable.list))
invisible(dev.off())

print("Add total intensity and missing value information into the design table")
design.table.list =
  lapply(names(design.table.list), function(i) {
    design.table = design.table.list[[i]] %>%
      left_join(sum.list[[i]])
    design.table.filename = paste0(designP, "/design_", i, ".tsv")
    write.table(design.table, design.table.filename, quote=F, row.names=F, sep="\t")
    design.table
}) %>% setNames(names(design.table.list))

# design.table.list = lapply(c("CYTOSOLIC", "PELLET", "SYNAPTOSOMAL"), function(i) {
#   read_table(paste0(designP, "/design_", i, ".tsv"))
# }) %>% setNames(c("CYTOSOLIC", "PELLET", "SYNAPTOSOMAL"))

RINvsPH = function(table) {
  pl = ggplot(table, aes(x=RIN, y=pH)) +
    geom_point()
  vars = gsub("~", "", as.character(pl$mapping))
  names(vars) = names(pl$mapping)
  #pllm = lm(formula = pl$data[,vars["x"], drop=T] ~ pl$data[,vars["y"], drop=T])
  pl + geom_smooth(method = "lm", formula = "y~x") +
    annotate("text", x = Inf, y = -Inf, hjust = 1.2, vjust = -2,
             label = paste0("Correlation = ",
             cor(pl$data[,vars["x"], drop=T], pl$data[,vars["y"], drop=T], method = 'pears
               round(., 2)))
}

pdf(paste0(OUTDIR_QC, "/RINvsPH.pdf"), h=3, w=4)
RINvsPH(design.table.list$CYTOSOLIC)
dev.off()
RINvsPH(design.table.list$CYTOSOLIC)
```




```
[1] "Get sample names"
[1] "Import all three datasets together"
[1] "Here we start from the company filtered raw data"
[1] "Those are supposed to be the intensity corrected"
[1] "Those are supposed to be the intensity corrected"
[1] "Those are supposed to be the intensity corrected"
[1] "Those are supposed to be the intensity corrected"
[1] "Those are supposed to be the intensity corrected"
[1] "Those are supposed to be the intensity corrected"
 [1] "The column names of the dataset CYTOSOLIC is Protein_IDs"
 [2] "The column names of the dataset CYTOSOLIC is Gene_names"
 [3] "The column names of the dataset CYTOSOLIC is 126_G1"
 [4] "The column names of the dataset CYTOSOLIC is 127N_G1"
 [5] "The column names of the dataset CYTOSOLIC is 127C_G1"
 [6] "The column names of the dataset CYTOSOLIC is 128N_G1"
 [7] "The column names of the dataset CYTOSOLIC is 128C_G1"
 [8] "The column names of the dataset CYTOSOLIC is 129N_G1"
 [9] "The column names of the dataset CYTOSOLIC is 129C_G1"
[10] "The column names of the dataset CYTOSOLIC is 130N_G1"
[11] "The column names of the dataset CYTOSOLIC is 130C_G1"
[12] "The column names of the dataset CYTOSOLIC is 131N_G1"
[13] "The column names of the dataset CYTOSOLIC is 131C_G1"
[14] "The column names of the dataset CYTOSOLIC is 132N_G1"
[15] "The column names of the dataset CYTOSOLIC is 132C_G1"
[16] "The column names of the dataset CYTOSOLIC is 126_G2"
[17] "The column names of the dataset CYTOSOLIC is 128N_G2"
[18] "The column names of the dataset CYTOSOLIC is 128C_G2"
[19] "The column names of the dataset CYTOSOLIC is 129N_G2"
```

```
[20] "The column names of the dataset CYTOSOLIC is 129C_G2"
[21] "The column names of the dataset CYTOSOLIC is 130N_G2"
[22] "The column names of the dataset CYTOSOLIC is 130C_G2"
[23] "The column names of the dataset CYTOSOLIC is 131N_G2"
[24] "The column names of the dataset CYTOSOLIC is 131C_G2"
[25] "The column names of the dataset CYTOSOLIC is 132N_G2"
[26] "The column names of the dataset CYTOSOLIC is 132C_G2"
[27] "The column names of the dataset CYTOSOLIC is 133N_G2"
[28] "The column names of the dataset CYTOSOLIC is 133C_G2"
[1] "The original number of proteins for the dataset CYTOSOLIC is 4083"
 [1] "The column names of the dataset PELLET is Protein_IDs"
 [2] "The column names of the dataset PELLET is Gene_names"
 [3] "The column names of the dataset PELLET is 127N_G1"
 [4] "The column names of the dataset PELLET is 127C_G1"
 [5] "The column names of the dataset PELLET is 128N_G1"
 [6] "The column names of the dataset PELLET is 128C_G1"
 [7] "The column names of the dataset PELLET is 129N_G1"
 [8] "The column names of the dataset PELLET is 129C_G1"
 [9] "The column names of the dataset PELLET is 130N_G1"
[10] "The column names of the dataset PELLET is 130C_G1"
[11] "The column names of the dataset PELLET is 131N_G1"
[12] "The column names of the dataset PELLET is 131C_G1"
[13] "The column names of the dataset PELLET is 132N_G1"
[14] "The column names of the dataset PELLET is 132C_G1"
[15] "The column names of the dataset PELLET is 134C_G1"
[16] "The column names of the dataset PELLET is 128N_G2"
[17] "The column names of the dataset PELLET is 128C_G2"
[18] "The column names of the dataset PELLET is 129N_G2"
[19] "The column names of the dataset PELLET is 129C_G2"
[20] "The column names of the dataset PELLET is 130N_G2"
[21] "The column names of the dataset PELLET is 130C_G2"
[22] "The column names of the dataset PELLET is 131N_G2"
[23] "The column names of the dataset PELLET is 131C_G2"
[24] "The column names of the dataset PELLET is 132N_G2"
[25] "The column names of the dataset PELLET is 132C_G2"
[26] "The column names of the dataset PELLET is 133N_G2"
[27] "The column names of the dataset PELLET is 133C_G2"
[28] "The column names of the dataset PELLET is 134C_G2"
[1] "The original number of proteins for the dataset PELLET is 4373"
 [1] "The column names of the dataset SYNAPTOSOMAL is Protein_IDs"
 [2] "The column names of the dataset SYNAPTOSOMAL is Gene_names"
 [3] "The column names of the dataset SYNAPTOSOMAL is 127N_G1"
 [4] "The column names of the dataset SYNAPTOSOMAL is 127C_G1"
 [5] "The column names of the dataset SYNAPTOSOMAL is 128N_G1"
 [6] "The column names of the dataset SYNAPTOSOMAL is 128C_G1"
```

```
 [7] "The column names of the dataset SYNAPTOSOMAL is 129N_G1"
 [8] "The column names of the dataset SYNAPTOSOMAL is 129C_G1"
 [9] "The column names of the dataset SYNAPTOSOMAL is 130N_G1"
[10] "The column names of the dataset SYNAPTOSOMAL is 130C_G1"
[11] "The column names of the dataset SYNAPTOSOMAL is 131N_G1"
[12] "The column names of the dataset SYNAPTOSOMAL is 131C_G1"
[13] "The column names of the dataset SYNAPTOSOMAL is 132N_G1"
[14] "The column names of the dataset SYNAPTOSOMAL is 132C_G1"
[15] "The column names of the dataset SYNAPTOSOMAL is 134N_G1"
[16] "The column names of the dataset SYNAPTOSOMAL is 128N_G2"
[17] "The column names of the dataset SYNAPTOSOMAL is 128C_G2"
[18] "The column names of the dataset SYNAPTOSOMAL is 129N_G2"
[19] "The column names of the dataset SYNAPTOSOMAL is 129C_G2"
[20] "The column names of the dataset SYNAPTOSOMAL is 130N_G2"
[21] "The column names of the dataset SYNAPTOSOMAL is 130C_G2"
[22] "The column names of the dataset SYNAPTOSOMAL is 131N_G2"
[23] "The column names of the dataset SYNAPTOSOMAL is 131C_G2"
[24] "The column names of the dataset SYNAPTOSOMAL is 132N_G2"
[25] "The column names of the dataset SYNAPTOSOMAL is 132C_G2"
[26] "The column names of the dataset SYNAPTOSOMAL is 133N_G2"
[27] "The column names of the dataset SYNAPTOSOMAL is 133C_G2"
[28] "The column names of the dataset SYNAPTOSOMAL is 134N_G2"
[1] "The original number of proteins for the dataset SYNAPTOSOMAL is 5183"
[[1]]
[1] "After removing NA-containing proteins, the number of proteins for the dataset CYTOSOLIC

[[2]]
[1] "After removing NA-containing proteins, the number of proteins for the dataset PELLET is

[[3]]
[1] "After removing NA-containing proteins, the number of proteins for the dataset SYNAPTOSO

[1] "There are lots of NA-containing proteins, "
[1] "if we remove them, we will remove lots of info, so I will keep them"
[1] "I also notice that compared to the DEP table (sheet5) the company used,"
[1] "there are extra samples, which represent pool samples, we will use them to calculate ra
[1] "Add total intensity and missing value information into the design table"
pdf
  2
```

```r
print("Make the design table with the ref info...")

names(design.table.list)
names(dataTable.list)
```

```r

design.table.list2 = lapply(seq_along(dataTable.list), function(i) {
  t = dataTable.list[[i]]
  d = design.table.list[[i]]
  samples = setdiff(colnames(t), c("Protein_IDs", "Gene_names"))
  ref = setdiff(samples, d$sample)
  d[c(nrow(d)+1, nrow(d)+2),] =
    as.data.frame(list(sample=ref,
    Batches=sapply(ref, function(x) strsplit(x, "_")[[1]][2]),
    Fraction=d$Fraction[1], Condition="ref"))
  design.table.filename = paste0(designP, "/design_", sample_names[i], "_wRef.tsv")
  write.table(d, design.table.filename, quote=F, row.names=F, sep="\t")
  d
})  %>% setNames(sample_names)
```

```
[1] "Make the design table with the ref info..."
[1] "CYTOSOLIC"    "PELLET"       "SYNAPTOSOMAL"
[1] "CYTOSOLIC"    "PELLET"       "SYNAPTOSOMAL"
```

4 Normalization

```r
pdf(paste0(OUTDIR_QC, "/Boxplot_raw.pdf"), h=4, w=10)
for (i in names(dataTable.list)) {
  data_matrix = dataTable.list[[i]][3:ncol(dataTable.list[[i]])] %>%
    as.matrix()
  boxplot(log2(data_matrix), main=paste0(i, "_Raw data_boxplot"),
          ylab="log2 scale")
  plotRLE(data_matrix, main=paste0(i, "_Raw data_RLE"),
          ylab="log2 scale")
  plotDensities(log2(data_matrix), main = paste0(i, "_Raw data_density"),
          ylab="log2 scale")
}
invisible(dev.off())

print("Sample loading (SL) normalization")
print("--Normalize on total abundance...")
dataTable.list_SL = lapply(names(dataTable.list), function(i){
  data_matrix = dataTable.list[[i]]
  design_matrix = design.table.list2[[i]]
  TMTset = design_matrix$Batch=="G1"
  data_matrix[,design_matrix[TMTset, "sample"]] <-
    sweep(data_matrix[,design_matrix[TMTset, "sample"]], 2,
```

```r
          mean(colSums(data_matrix[,design_matrix[TMTset, "sample"]], na.rm = T)) /
            colSums(data_matrix[,design_matrix[TMTset, "sample"]],
                    na.rm = T), FUN = "*")
  TMTset = design_matrix$Batch=="G2"
  data_matrix[,design_matrix[TMTset, "sample"]] <-
    sweep(data_matrix[,design_matrix[TMTset, "sample"]], 2,
          mean(colSums(data_matrix[,design_matrix[TMTset, "sample"]], na.rm = T)) /
            colSums(data_matrix[,design_matrix[TMTset, "sample"]],
                    na.rm = T), FUN = "*")

  data_matrix
}) %>% setNames(names(dataTable.list))

print("Check whether each batch do have the same total abundance:")
i = 1
colSums(dataTable.list_SL[[i]][,3:ncol(dataTable.list_SL[[i]])], na.rm=T)
i = 2
colSums(dataTable.list_SL[[i]][,3:ncol(dataTable.list_SL[[i]])], na.rm=T)
i = 3
colSums(dataTable.list_SL[[i]][,3:ncol(dataTable.list_SL[[i]])], na.rm=T)

pdf(paste0(OUTDIR_QC, "/Boxplot_SL.pdf"), h=4, w=10)
for (i in names(dataTable.list_SL)) {
  data_matrix = dataTable.list_SL[[i]][3:ncol(dataTable.list_SL[[i]])] %>%
    as.matrix()
  boxplot(log2(data_matrix), main=paste0(i, "_SL_boxplot"),
          ylab="log2 scale")
  plotRLE(data_matrix, main=paste0(i, "_SL_RLE"),
          ylab="log2 scale")
  plotDensities(log2(data_matrix), main = paste0(i, "_SL_density"),
          ylab="log2 scale")
}
invisible(dev.off())
```

```
[1] "Sample loading (SL) normalization"
[1] "--Normalize on total abundance..."
[1] "Check whether each batch do have the same total abundance:"
     126_G1     127N_G1     127C_G1     128N_G1     128C_G1     129N_G1
13891412466 13891412466 13891412466 13891412466 13891412466 13891412466
    129C_G1     130N_G1     130C_G1     131N_G1     131C_G1     132N_G1
13891412466 13891412466 13891412466 13891412466 13891412466 13891412466
    132C_G1      126_G2     128N_G2     128C_G2     129N_G2     129C_G2
13891412466 11966254471 11966254471 11966254471 11966254471 11966254471
    130N_G2     130C_G2     131N_G2     131C_G2     132N_G2     132C_G2
```

```
11966254471 11966254471 11966254471 11966254471 11966254471 11966254471
    133N_G2     133C_G2
11966254471 11966254471
    127N_G1     127C_G1     128N_G1     128C_G1     129N_G1     129C_G1
 7666703282  7666703282  7666703282  7666703282  7666703282  7666703282
    130N_G1     130C_G1     131N_G1     131C_G1     132N_G1     132C_G1
 7666703282  7666703282  7666703282  7666703282  7666703282  7666703282
    134C_G1     128N_G2     128C_G2     129N_G2     129C_G2     130N_G2
 7666703282 13452516187 13452516187 13452516187 13452516187 13452516187
    130C_G2     131N_G2     131C_G2     132N_G2     132C_G2     133N_G2
13452516187 13452516187 13452516187 13452516187 13452516187 13452516187
    133C_G2     134C_G2
13452516187 13452516187
    127N_G1     127C_G1     128N_G1     128C_G1     129N_G1     129C_G1
25432229346 25432229346 25432229346 25432229346 25432229346 25432229346
    130N_G1     130C_G1     131N_G1     131C_G1     132N_G1     132C_G1
25432229346 25432229346 25432229346 25432229346 25432229346 25432229346
    134N_G1     128N_G2     128C_G2     129N_G2     129C_G2     130N_G2
25432229346 30372460083 30372460083 30372460083 30372460083 30372460083
    130C_G2     131N_G2     131C_G2     132N_G2     132C_G2     133N_G2
30372460083 30372460083 30372460083 30372460083 30372460083 30372460083
    133C_G2     134N_G2
30372460083 30372460083
```

```r
print("Internal Reference Scaling (IRS) normalization")
print("Grab the ref sample names...")
ref.names = mapply(function(i, j) {
  setdiff(j$sample, i$sample)
}, design.table.list, design.table.list2)

dataTable.list_SL_IRS = lapply(names(dataTable.list_SL), function(i) {
  data_matrix = dataTable.list_SL[[i]]
  ref = ref.names[, i]
  design_matrix = design.table.list2[[i]]
  # refScaler<-apply(data_matrix[, ref], 1,
  #                       function(x) exp(mean(log(x), na.rm=T)))
  print("Change zero value to NA...")
  data_matrix = replace(data_matrix, data_matrix==0, NA)
  print("We will scale the ratio up by the geometric mean of the 2 ref samples for each ge
  refScaler=apply(data_matrix[,ref], 1, function(x) exp(mean(log(x),na.rm=T)))
  # refScaler=exp(mean(log(data_matrix%>% na.omit() %>%
  #                 .[, 3:ncol(data_matrix)] %>%
  #                 apply(., 2, function(x) exp(mean(log(x)))))))
  TMTsetSample = design_matrix[design_matrix$Batch=="G1", "sample"]
```

```
  print("There are lots of NA ref genes , will not calculate the ratios for those genes")
  print(sum(is.na(data_matrix[, ref[1]])))
  nonNAref = !is.na(data_matrix[, ref[1]])
  data_matrix[nonNAref,TMTsetSample]<-
    data_matrix[nonNAref,TMTsetSample]*(refScaler[nonNAref]/data_matrix[nonNAref,ref[1]])
  TMTsetSample = design_matrix[design_matrix$Batch=="G2", "sample"]
  print("There are lots of NA ref genes , will not calculate the ratios for those genes")
  print(sum(is.na(data_matrix[, ref[2]])))
  nonNAref = !is.na(data_matrix[, ref[2]])
  data_matrix[nonNAref,TMTsetSample]<-
    data_matrix[nonNAref,TMTsetSample]*(refScaler[nonNAref]/data_matrix[nonNAref,ref[2]])
  data_matrix
}) %>% setNames(names(dataTable.list_SL))

print("Compare the normalized results:")
# head(dataTable.list_SL_IRS[[1]])
# head(dataTable.list_SL[[1]])
# head(dataTable.list[[1]])

pdf(paste0(OUTDIR_QC, "/Boxplot_SL_IRS.pdf"), h=4, w=10)
for (i in names(dataTable.list_SL_IRS)) {
  data_matrix = dataTable.list_SL_IRS[[i]][3:ncol(dataTable.list_SL_IRS[[i]])] %>%
    as.matrix()
  boxplot(log2(data_matrix), main=paste0(i, "_IRS_boxplot"),
          ylab="log2 scale")
  plotRLE(data_matrix, main=paste0(i, "_IRS_RLE"),
          ylab="log2 scale")
  plotDensities(log2(data_matrix), main = paste0(i, "_IRS_density"),
          ylab="log2 scale")
}
invisible(dev.off())

print("Transform and clean...")
dataTable.list_SL_IRS_clean = lapply(names(dataTable.list_SL_IRS), function(i){
  data_matrix = dataTable.list_SL_IRS[[i]]
  ref = ref.names[, i]
  data_matrix = data_matrix[, !colnames(data_matrix)%in%ref]
  # sample_names = grep("_G", colnames(data_matrix), value=T)
  # data_matrix[, sample_names] = log2(data_matrix[, sample_names])
  # data_matrix[, sample_names] =
  #   #limma::normalizeCyclicLoess(data_matrix[, sample_names])
  #   limma::normalizeMedianValues(data_matrix[, sample_names])
  data_matrix
}) %>% setNames(names(dataTable.list_SL_IRS))
```

```
[1] "Internal Reference Scaling (IRS) normalization"
[1] "Grab the ref sample names..."
[1] "Change zero value to NA..."
[1] "We will scale the ratio up by the geometric mean of the 2 ref samples for each gene"
[1] "There are lots of NA ref genes , will not calculate the ratios for those genes"
[1] 319
[1] "There are lots of NA ref genes , will not calculate the ratios for those genes"
[1] 545
[1] "Change zero value to NA..."
[1] "We will scale the ratio up by the geometric mean of the 2 ref samples for each gene"
[1] "There are lots of NA ref genes , will not calculate the ratios for those genes"
[1] 568
[1] "There are lots of NA ref genes , will not calculate the ratios for those genes"
[1] 700
[1] "Change zero value to NA..."
[1] "We will scale the ratio up by the geometric mean of the 2 ref samples for each gene"
[1] "There are lots of NA ref genes , will not calculate the ratios for those genes"
[1] 689
[1] "There are lots of NA ref genes , will not calculate the ratios for those genes"
[1] 390
[1] "Compare the normalized results:"
[1] "Transform and clean..."
```

```r
print("TMM normalization")
# In order to calculate TMM normalization factor, we need to replace NA with 0 first:
dataTable.list_SL_IRS_TMM =
  lapply(names(dataTable.list_SL_IRS_clean), function(i){
  data_matrix = dataTable.list_SL_IRS_clean[[i]]
  sample_names = grep("_G", colnames(data_matrix), value=T)
  temp =
    replace(data_matrix[,sample_names], is.na(data_matrix[,sample_names]), 0)
  TMM = calcNormFactors(temp, method = "TMMwsp")
  data_matrix[,sample_names] =
    sweep(data_matrix[,sample_names], 2, TMM, FUN = "/") %>%
    log2()
  data_matrix
}) %>% setNames(names(dataTable.list_SL_IRS_clean))

pdf(paste0(OUTDIR_QC, "/Boxplot_SL_IRS_clean_TMM.pdf"), h=4, w=10)
for (i in names(dataTable.list_SL_IRS_TMM)) {
  data_matrix =
    dataTable.list_SL_IRS_TMM[[i]]
  sample_names = grep("_G", colnames(data_matrix), value=T)
  data_matrix = data_matrix[, sample_names] %>% as.matrix()
```

```
  boxplot(data_matrix, main=paste0(i, "_TMM_boxplot"),
          ylab="log2 scale")
  plotRLE(2^data_matrix, main=paste0(i, "_TMM_RLE"),
          ylab="log2 scale")
  plotDensities(data_matrix, main = paste0(i, "_TMM_density"),
          ylab="log2 scale")
}
invisible(dev.off())

print("Check how many proteins are completely missing:")
lapply(dataTable.list, function(d) {
  sample.names = grep("_G", colnames(d))
  nas = d[, sample.names] %>% is.na() %>%
    rowSums()
  sum(nas==24)
})

lapply(dataTable.list_SL, function(d) {
  sample.names = grep("_G", colnames(d))
  nas = d[, sample.names] %>% is.na() %>%
    rowSums()
  sum(nas==24)
})

lapply(dataTable.list_SL_IRS, function(d) {
  sample.names = grep("_G", colnames(d))
  nas = d[, sample.names] %>% is.na() %>%
    rowSums()
  sum(nas==24)
})

lapply(dataTable.list_SL_IRS_TMM, function(d) {
  sample.names = grep("_G", colnames(d))
  nas = d[, sample.names] %>% is.na() %>%
    rowSums()
  sum(nas==24)
})

dataTable.list_SL_IRS_TMM=lapply(dataTable.list_SL_IRS_TMM,function(d){
  sample.names = grep("_G", colnames(d))
  nas = d[, sample.names] %>% is.na() %>%
    rowSums()
  d[!(nas==24), ]
})
```

```r

lapply(dataTable.list_SL_IRS_TMM, function(d) {
  sample.names = grep("_G", colnames(d))
  nas = d[, sample.names] %>% is.na() %>%
    rowSums()
  sum(nas==24)
})

lapply(names(dataTable.list_SL_IRS_TMM), function(i) {
  print(i)
  d = dataTable.list_SL_IRS_TMM[[i]]
  print(paste0("Check the dim of normalized count table for ", i))
  print(dim(d))
  print(paste0("Check the dim of the normalized count table after na.omit for ", i))
  print(dim(na.omit(d)))
})
```

```
[1] "TMM normalization"
[1] "Check how many proteins are completely missing:"
$CYTOSOLIC
[1] 0

$PELLET
[1] 0

$SYNAPTOSOMAL
[1] 0

$CYTOSOLIC
[1] 0

$PELLET
[1] 0

$SYNAPTOSOMAL
[1] 0

$CYTOSOLIC
[1] 0

$PELLET
[1] 0

$SYNAPTOSOMAL
```

```
[1] 0

$CYTOSOLIC
[1] 0

$PELLET
[1] 1

$SYNAPTOSOMAL
[1] 0

$CYTOSOLIC
[1] 0

$PELLET
[1] 0

$SYNAPTOSOMAL
[1] 0

[1] "CYTOSOLIC"
[1] "Check the dim of normalized count table for CYTOSOLIC"
[1] 4083   26
[1] "Check the dim of the normalized count table after na.omit for CYTOSOLIC"
[1] 3220   26
[1] "PELLET"
[1] "Check the dim of normalized count table for PELLET"
[1] 4372   26
[1] "Check the dim of the normalized count table after na.omit for PELLET"
[1] 3106   26
[1] "SYNAPTOSOMAL"
[1] "Check the dim of normalized count table for SYNAPTOSOMAL"
[1] 5183   26
[1] "Check the dim of the normalized count table after na.omit for SYNAPTOSOMAL"
[1] 4104   26
[[1]]
[1] 3220   26

[[2]]
[1] 3106   26

[[3]]
[1] 4104   26
```

```r
print("Sort out gene names...")
#print("Some proteins have two genes, we will only keep the 1st one if it is in the gene S
print("Some genes are obsolete, we will update it...")

#suppressMessages(library(org.Hs.eg.db))
symb <- keys(org.Hs.eg.db, "SYMBOL")

dataTable.list_SL_IRS_TMM = lapply(dataTable.list_SL_IRS_TMM, function(dat) {
  print("Remove duplicated (gene names for the same protein ID")
  dat$Gene_names = dat$Gene_names %>% sapply(., function(x) {
    strsplit(x, ";") %>% unlist() %>%
      unique() %>% paste(., collapse =";")
  })
  print("Change gene names to updated human gene symbols")
  INDEX = !(dat$Gene_names %in% symb)
  temp = dat$Gene_names[INDEX]
  temp2 = sapply(temp, function(i) {
    gene.list = strsplit(i, ";")[[1]]
    gene.list = limma::alias2Symbol(c(gene.list))
    gene.list[1]
  })
  dat$Genes = dat$Gene_names
  dat = dat %>%
    dplyr::select(Protein_IDs, Genes, Gene_names, everything())
  dat$Genes[INDEX] = temp2
  dat
  print("Fix empty gene names")
  INDEX.NA = is.na(dat$Genes)
  #k <- keys(UniProt.ws, "UNIPROTKB")
  query = dat[INDEX.NA, "Protein_IDs"] #%>% setdiff(., c("Q00169", "Q9BRA0"))
  # library(UniProt.ws)
  Gene.Names =
    mapUniProt("UniProtKB_AC-ID", "UniProtKB", query = query) %>%
    .[, "Gene.Names"]
  new.Gene.Names = if (sum(!is.na(Gene.Names)) > 0) {
      hgncs = sapply(Gene.Names, function(genes) {
        if (genes != "") {
          gene.list = strsplit(genes, " ")[[1]]
          hgnc = gene.list[gene.list %in% symb]
          hgnc[1]
        }
      })
    } %>% unlist() %>% as.character()
  dat[INDEX.NA, "Genes"] = new.Gene.Names
```

```
  dat
})

# library(UniProt.ws)
# up <- UniProt.ws(taxId=9606)
```

```
[1] "Sort out gene names..."
[1] "Some genes are obsolete, we will update it..."
[1] "Remove duplicated (gene names for the same protein ID"
[1] "Change gene names to updated human gene symbols"
[1] "Fix empty gene names"
[1] "Remove duplicated (gene names for the same protein ID"
[1] "Change gene names to updated human gene symbols"
[1] "Fix empty gene names"
[1] "Remove duplicated (gene names for the same protein ID"
[1] "Change gene names to updated human gene symbols"
[1] "Fix empty gene names"
```

```
print("Check Protein IDs...")
# i = 1
# suppressMessages(library(UniProt.ws))
# library(rbioapi)
# source("../src/getNewUniprotID.r")
# uniprotID = getNewUniprotID(dataTable.list_corrrected[[i]]$Protein_IDs)
# length(uniprotID)
# dim(dataTable.list_corrrected[[i]])
```

```
[1] "Check Protein IDs..."
```

5 QC

5.1 PCA

```
makePCAplot =
  function(dataTable.list_SL_IRS_TMM, design.table.list,
           DEGs=NULL, onlyDEGs =FALSE){
    invisible(lapply(names(dataTable.list_SL_IRS_TMM), function(i) {
      if (onlyDEGs) {
        data_matrix = dataTable.list_SL_IRS_TMM[[i]] %>%
          filter(Protein_IDs %in% DEGtops[[i]])
      } else {
```

```
        data_matrix = dataTable.list_SL_IRS_TMM[[i]]
      }
      sample_names = grep("_G", colnames(data_matrix), value=T)
      rownames(data_matrix) = data_matrix$Protein_IDs
      c_pca = prcomp(t(na.omit(data_matrix[,sample_names])),
                     scale=TRUE, center = TRUE)
      frac_var = function(x) x^2/sum(x^2)
      design = design.table.list[[i]]
      design$Age_group = ifelse(design$Age<=45, "0-45",
                                ifelse(design$Age<=55, "46-55", "56-65"))
      var_explained = c_pca$sdev %>%
        tibble::as_tibble() %>%
        frac_var() %>%
        round(digits = 2)
      c_pca_all <- c_pca$x %>%
        as.data.frame() %>%
        tibble::rownames_to_column(var = "sample") %>%
        dplyr::full_join(design, by = "sample")
      #library(GGally)
      P1=suppressMessages(ggpairs(c_pca_all,
                                  columns=c('PC1', 'PC2', 'PC3', "PC4", "Batch", "Fraction
                                            "Condition", "Sex", "Age", "RIN", "PMI",
                                            "Total_intensity", "MissingValues"),
                                  mapping=aes(color=Batch), upper=list(continuous='blank')
                                  main="Colored by Batch"))
      P2=suppressMessages(ggpairs(c_pca_all,
                                  columns=c('PC1', 'PC2', 'PC3', "PC4", "Batch", "Fraction
                                            "Condition", "Sex", "Age", "RIN", "PMI",
                                            "Total_intensity", "MissingValues"),
                                  mapping=aes(color=Condition), upper=list(continuous='bla
                                  main="Colored by Condition"))
      FILE = ifelse(onlyDEGs, paste0(OUTDIR_QC, "/", i, "_PCA_plot_onDEGs.pdf"),
                    paste0(OUTDIR_QC, "/", i, "_PCA_plot.pdf"))
      pdf(FILE, h=7.5, w=10)
      print(P1)
      print(P2)

      vars = list(c("Batch", "Condition"),
                  c("Condition", "RIN"),
                  c("Condition", "Sex"),
                  c("Condition", "PMI"),
                  c("Condition", "Age"),
                  c("Condition", "Total_intensity"),
                  c("Condition", "MissingValues"),
```

```
                  c("Condition", "pH"),
                  c("Condition", "TissueStorageTime_months"))
      lapply(vars, function(V) {
        p3 <- ggplot(aes(PC1, PC2), data = c_pca_all) +
          geom_point(size = 3, aes_string(shape = V[1], col=V[2])) +
          labs(x = paste("PC1 (", var_explained[1,"value"]*100, "% explained var.)", sep =
               y = paste("PC2 (", var_explained[2,"value"]*100, "% explained var.)", sep =
        p4 <- ggplot(aes(PC1, PC3), data = c_pca_all) +
          geom_point(size = 3, aes_string(shape = V[1], col=V[2])) +
          labs(x = paste("PC1 (", var_explained[1,"value"]*100, "% explained var.)", sep =
               y = paste("PC3 (", var_explained[3,"value"]*100, "% explained var.)", sep =
        p5 <- ggplot(aes(PC3, PC2), data = c_pca_all) +
          geom_point(size = 3, aes_string(shape = V[1], col=V[2])) +
          labs(x = paste("PC3 (", var_explained[3,"value"]*100, "% explained var.)", sep =
               y = paste("PC2 (", var_explained[2,"value"]*100, "% explained var.)", sep =
        p52 <- ggplot(aes(PC2, PC4), data = c_pca_all) +
          geom_point(size = 3, aes_string(shape = V[1], col=V[2])) +
          labs(x = paste("PC2 (", var_explained[2,"value"]*100, "% explained var.)", sep =
               y = paste("PC4 (", var_explained[4,"value"]*100, "% explained var.)", sep =
        print(cowplot::plot_grid(p3, p4, p5, p52, ncol=2, labels=i))
      })
      invisible(dev.off())
  }))
}

makePCAplot(dataTable.list_SL_IRS_TMM, design.table.list,
            DEGs=NULL, onlyDEGs =FALSE)
```

5.2 Outlier

From the prelimiary PCA plots, we know that cytosolic comparment may have an outlier, here we will have a check

```
# Outlier detection by the Mahalanobis distances of PC1 and PC2. Also plot PC1 and PC2 wit
data_matrix = dataTable.list_SL_IRS_TMM[["CYTOSOLIC"]]
sample_names = grep("_G", colnames(data_matrix), value=T)
rownames(data_matrix) = data_matrix$Protein_IDs
p = mt::pca.outlier(t(na.omit(data_matrix[,sample_names])))
pdf(paste0(OUTDIR_QC, "/PCA_plot_outlier_CYTOSOL.pdf"), h=4, w=4)
p
dev.off()

print("After checking the meta data table, we can see that this sample has the lowest RIN
```

```r
print(", indicating a compromised sample quality")
print("so we will remove the sample...")
dataTable.list_SL_IRS_TMM[["CYTOSOLIC"]] =
  dataTable.list_SL_IRS_TMM[["CYTOSOLIC"]][,setdiff(colnames(dataTable.list_SL_IRS_TMM[["C
                                                  "133C_G2")]

print("Remove this sample from the design table as well...")
design.table.list$CYTOSOLIC =
  design.table.list$CYTOSOLIC %>% filter(!(sample %in% "133C_G2"))

# Save the normalized counts
saveRDS(dataTable.list_SL_IRS_TMM,
        paste0(OUTDIR_QC, "/NormalizedIntensity_dataTable.list_SL_IRS_TMM.RDS"))
# dataTable.list_SL_IRS_TMM =
#   readRDS(paste0(OUTDIR_QC, "/NormalizedIntensity_dataTable.list_SL_IRS_TMM.RDS"))

rm(data_matrix, dataTable.list, dataTable.list0, dataTable.list_SL,
   dataTable.list_SL_IRS, dataTable.list_SL_IRS_clean)
options(java.parameters = "-Xmx1000m")
if (file.exists(paste0(OUTDIR_QC, "NormalizedIntensity.xlsx"))) {
  file.remove(paste0(OUTDIR_QC, "NormalizedIntensity.xlsx"))
}
invisible(lapply(seq_along(dataTable.list_SL_IRS_TMM), function(i) {
    options(java.parameters = "-Xmx3000m")
    write.xlsx(as.data.frame(dataTable.list_SL_IRS_TMM[[i]]),
             paste0(OUTDIR_QC, "NormalizedIntensity.xlsx"),
             sheetName = names(dataTable.list_SL_IRS_TMM)[i],
             col.names=TRUE, row.names=FALSE,
             append=T)}))
```

```
$plot

$outlier
133C_G2
     24

$conf.level
[1] 0.975

$mah.dist
  127N_G1   127C_G1   128N_G1   128C_G1   129N_G1   129C_G1   130N_G1   130C_G1
1.8938920 1.1901082 1.0623803 0.5270901 0.5473020 0.8221021 1.4925359 0.9323676
  131N_G1   131C_G1   132N_G1   132C_G1   128N_G2   128C_G2   129N_G2   129C_G2
2.1062988 1.4205643 0.7681140 0.8536911 1.0011653 0.4836397 0.4605503 0.4925343
```

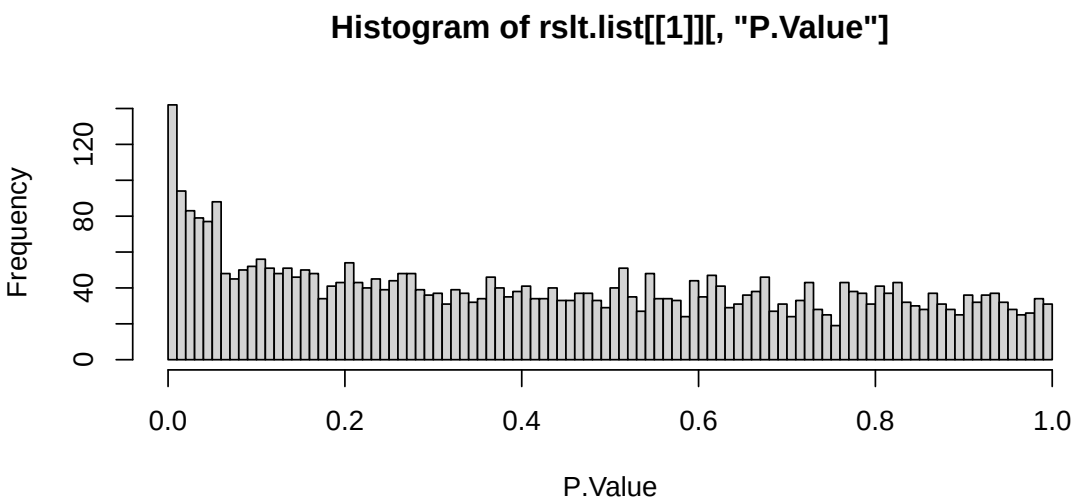
```
  130N_G2   130C_G2   131N_G2   131C_G2   132N_G2   132C_G2   133N_G2   133C_G2
1.1757581 0.9657251 0.8310979 0.5051868 0.5429451 1.1281854 1.8428660 4.2224446

$cutoff
[1] 2.716203

pdf
  2
[1] "After checking the meta data table, we can see that this sample has the lowest RIN valu
[1] ", indicating a compromised sample quality"
[1] "so we will remove the sample..."
[1] "Remove this sample from the design table as well..."
[1] TRUE
```

6 Statistical test

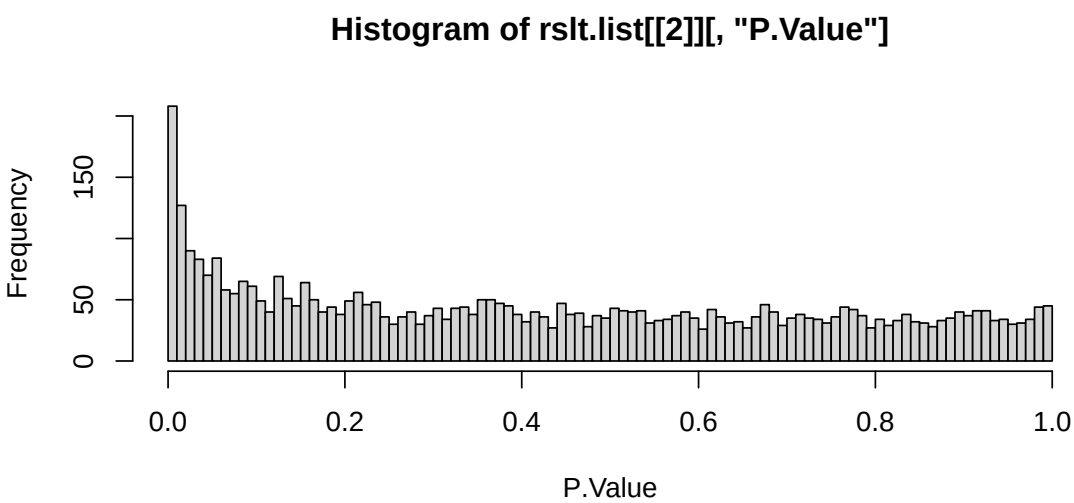
LIMMA comparisons using the normalized data

```r
# do parallel model fitting
rslt.list = lapply(names(dataTable.list_SL_IRS_TMM), function(i) {
  data_matrix = dataTable.list_SL_IRS_TMM[[i]]
  design_matrix = design.table.list[[i]]
  plan(multisession)
  models = future.apply::future_lapply(1:nrow(data_matrix),function(j){
    abundance = data_matrix[j, design_matrix$sample]
    tmp_data = data.frame(abundance=as.numeric(abundance),
                          Condition=design_matrix$Condition,
                          Batch=design_matrix$Batch,
                          RIN=as.numeric(design_matrix$RIN),
                          PMI=as.numeric(design_matrix$PMI),
                          Age=as.numeric(design_matrix$Age))
    tmp_data$Condition = factor(tmp_data$Condition,levels=c("control", "case"))
    if (i %in% c("CYTOSOLIC", "PELLET")) {
      #print("We will also control for RIN for CYTOSOLIC and PELLET")
      #print("Remove set for set specific proteins: ")
      if(any(tapply(is.na(abundance), tmp_data$Batch, all))){
        design = model.matrix(~0+Condition+RIN, data=tmp_data,verbose=FALSE)
      }else{
        design = model.matrix(~0+Condition+Batch+RIN,data=tmp_data,verbose=FALSE)
      }
    } else {
      if(any(tapply(is.na(abundance), tmp_data$Batch, all))){
        design = model.matrix(~0+Condition+Age, data=tmp_data,verbose=FALSE)
```

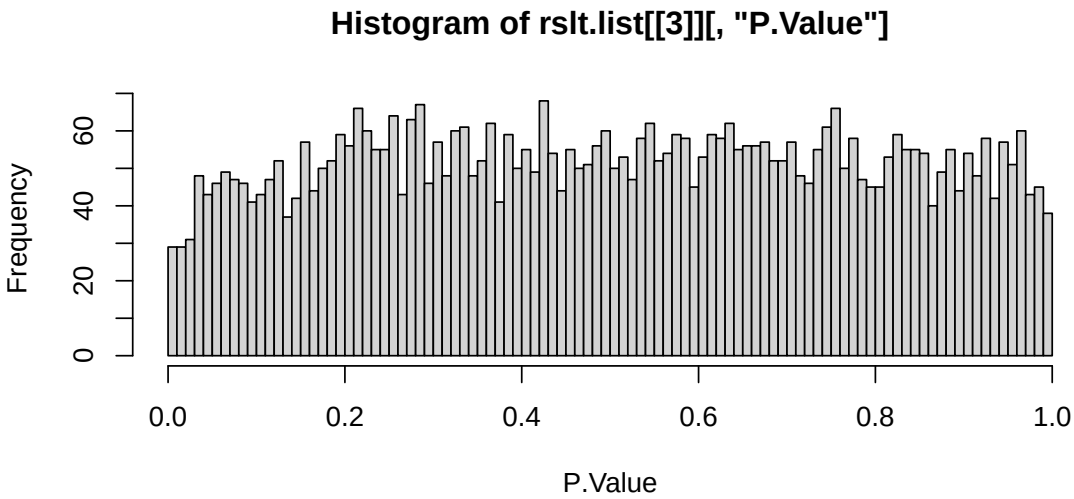
```
        # head(design)
        # Conditioncontrol Conditioncase Age
        # 1                1             0  52
        # 2                1             0  54
      }else{
        design = model.matrix(~0+Condition+Batch+Age,data=tmp_data,verbose=FALSE)
      }
    }
    contrast.matrix =
      makeContrasts(contrasts = "Conditioncase-Conditioncontrol",
                                                    levels = design)
    fit = lmFit(abundance, design=design)
    #Fit linear model for each gene
    fit2 = contrasts.fit(fit, contrast.matrix)
    #Given a linear model fit, compute estimated coefficients
    #and standard errors for a given set of contrasts.
    fit2 = eBayes(fit2, robust=T)
    # Given a linear model fit from lmFit, compute moderated t-statistics,
    #moderated F-statistic, and log-odds of differential expression by
    #empirical Bayes moderation of the standard errors towards a global value.
    limma_table = topTable(fit2,
                           coef = 1,
                           number = Inf,
                           sort.by = "none")
    limma_table
  })

  rslt = Reduce(rbind, models) %>% cbind(data_matrix) %>%
    dplyr::select(Protein_IDs, Genes, everything())
  rslt = arrange(rslt, P.Value)
  # reset index
  rownames(rslt) = NULL
  rslt$adj.P.Val = p.adjust(rslt$P.Value, method = "fdr")
  rslt
}) %>% setNames(names(dataTable.list_SL_IRS_TMM))

hist(rslt.list[[1]][, "P.Value"], breaks = 100, xlab = "P.Value")
```



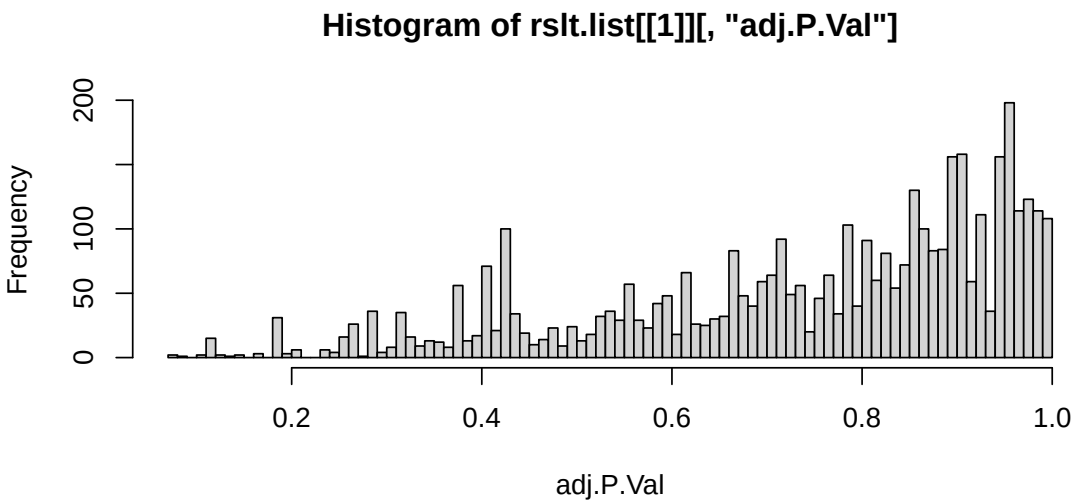

```r
hist(rslt.list[[2]][, "P.Value"], breaks = 100, xlab = "P.Value")
```



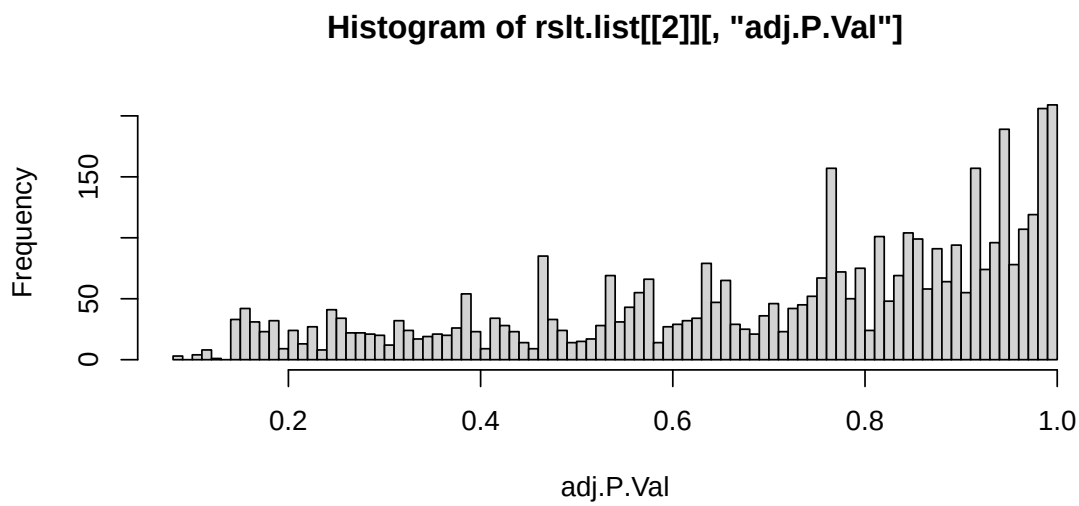

```r
hist(rslt.list[[3]][, "P.Value"], breaks = 100, xlab = "P.Value")
```



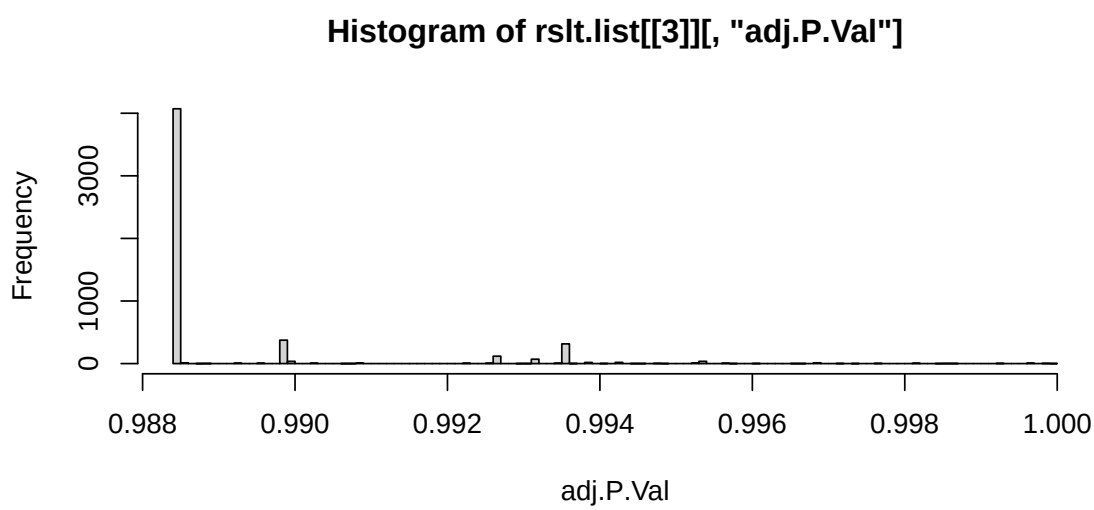

```
hist(rslt.list[[1]][, "adj.P.Val"], breaks = 100, xlab = "adj.P.Val")
```




```
hist(rslt.list[[2]][, "adj.P.Val"], breaks = 100, xlab = "adj.P.Val")
```




```
hist(rslt.list[[3]][, "adj.P.Val"], breaks = 100, xlab = "adj.P.Val")
```




```
#rslt.list[[1]] %>% filter(adj.P.Val <0.2)

sum(rslt.list[[1]]$adj.P.Val <0.2)
sum(rslt.list[[2]]$adj.P.Val <0.2)
sum(rslt.list[[3]]$adj.P.Val <0.2)

```

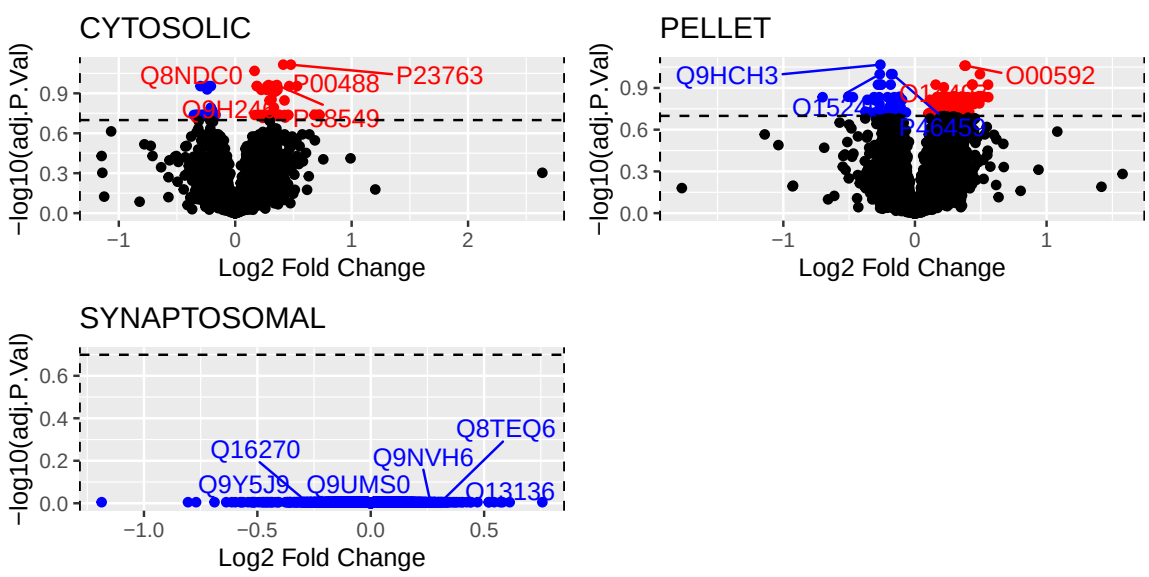
```r
sum(rslt.list[[1]]$adj.P.Val <0.1)
sum(rslt.list[[2]]$adj.P.Val <0.1)
sum(rslt.list[[3]]$adj.P.Val <0.1)

#save the full differential table
OUTDIR_DA = paste0(outDir, "/DifferentialAnalysis/")
dir.create(OUTDIR_DA, recursive = T, showWarnings = F)
lapply(names(rslt.list), function(i) {
  xlsx.file =
    paste0(OUTDIR_DA, "/Differential_Table_", i, ".xlsx")
  write.xlsx(rslt.list[[i]] %>% as.data.frame(),
    xlsx.file,
    sheetName = i, col.names = TRUE,
    row.names = F, append = F)
})

# rslt.list = lapply(names(dataTable.list_SL_IRS_TMM), function(i) {
#   read_excel(paste0(OUTDIR_DA, "/Differential_Table_", i, ".xlsx"),
#                  sheet=1, col_names=TRUE, na="", skip=0)
# }) %>% setNames(names(dataTable.list_SL_IRS_TMM))
```

```
[1] 62
[1] 186
[1] 0
[1] 3
[1] 3
[1] 0
[[1]]
NULL

[[2]]
NULL

[[3]]
NULL
```

Volcano plot

```r
doVolcano = function(resi, NAME, lfcShrink=F) {
  selectedCols <- c("Protein_IDs", "logFC", "P.Value", "adj.P.Val")
  toExport <- resi[, selectedCols]
  toExport <- toExport[order(toExport$P.Value),]

```

```r
  tmp <- as.data.frame(toExport) %>%
    filter(!is.na(adj.P.Val)) %>%
    mutate(sig = ifelse(adj.P.Val<0.2,
                        ifelse(logFC > 0, "up", "down"), "nonSig"))
  tmpSignif <- tmp %>%
    filter(P.Value < quantile(tmp$P.Value, 0.001))

  p = ggplot(data = tmp, aes(logFC, -log10(adj.P.Val), col = sig)) +
    geom_point() +
    xlab("Log2 Fold Change") +
    geom_text_repel(data = tmpSignif, aes(logFC, -log10(adj.P.Val),
                                          label = Protein_IDs)) +
    geom_hline(yintercept = -log10(0.2), lty = 2) +
    geom_vline(xintercept = c(-log2(0), log2(0)), lty = 2) +
    scale_colour_manual(values = c("blue", "black", "red")) +
    theme(legend.position="none") +
    ggtitle(ifelse(lfcShrink, paste0(NAME, "_lfcShrink"), paste0(NAME, "")))

  return(p)
}

NAMES = names(rslt.list)
for (i in seq_along(rslt.list)) {
  p = doVolcano(rslt.list[[i]], NAME=NAMES[i], lfcShrink=F)
  assign(paste0("P", i), p)
}
cowplot::plot_grid(P1, P2, P3, ncol=2, nrow=2)
```



```r
pdf(paste0(OUTDIR_DA, "Volcano_plot.pdf"), h=8, w=10)
cowplot::plot_grid(P1, P2, P3, ncol=2, nrow=2)
invisible(dev.off())
```

MA plot

```r
# dataTable.list_SL_IRS_TMM =
#   readRDS(paste0(OUTDIR_QC, "/NormalizedIntensity_dataTable.list_SL_IRS_TMM.RDS"))
#
# rslt.list = lapply(names(dataTable.list_SL_IRS_TMM), function(i) {
#   read_excel(paste0(OUTDIR_DA, "/Differential_Table_", i, ".xlsx"),
#                    sheet=1, col_names=TRUE, na="", skip=0)
#   }) %>% setNames(names(dataTable.list_SL_IRS_TMM))

pdf(paste0(OUTDIR_DA, "MA_plot.pdf"), h=3, w=3)
invisible(lapply(seq_along(rslt.list), function(i) {
  obj = rslt.list[[i]]
  name = names(rslt.list)[i]
  print(ggplot(obj, aes(x=AveExpr, y=logFC)) +
    geom_point() +
    geom_line(y=0, col="red") +
    ggtitle(paste0(name, "_MA plot")) +
    theme_bw())
}))
invisible(dev.off())
```

6.1 PCA on DEGs

NOTE: the batch effects were not removed from those PCA plots

```r
DEGtops = lapply(seq_along(rslt.list), function(d) {
  if (d<3) {
    rslt.list[[d]] %>% filter(adj.P.Val<0.2) %>%
      pull(Protein_IDs)
  } else {
    rslt.list[[d]] %>% top_n(n = 20, wt=adj.P.Val) %>%
      pull(Protein_IDs)
  }
}) %>% setNames(names(dataTable.list_SL_IRS_TMM))

makePCAplot(dataTable.list_SL_IRS_TMM, design.table.list,
            DEGs=DEGtops, onlyDEGs =TRUE)
```

6.2 Barplot of significant proteins abundance (adj.P.Val<0.2)

```
plotProteins = function(mat, design_matrix) {
  nCounts = mat %>% as.data.frame() %>%
    gather(-c(Protein_IDs,Genes, Gene_names, logFC,AveExpr,t,P.Value,adj.P.Val,B),
           key="sample", value="NormalizedAbundance") %>%
    filter(adj.P.Val<0.2) %>% full_join(design_matrix) %>%
    arrange(adj.P.Val, logFC)
  #library(plyr)
  df = ddply(nCounts, .(Batch, Condition, Protein_IDs), summarize,
             mean = round(mean(NormalizedAbundance), 2),
             sd = round(sd(NormalizedAbundance), 2)) #%>%
    #rename(.,c("NormalizedAbundance" = "mean"))
  #head(df)
  df$NormalizedAbundance = exp(df$mean)
  df$Protein_IDs = factor(df$Protein_IDs,
                          levels=unique(nCounts$Protein_IDs))

  n = length(unique(nCounts$Protein_IDs))
  theme_set(theme_bw())
  p = ggplot(df, aes(fill=Condition, y=NormalizedAbundance, x=Batch)) +
    geom_bar(position="dodge", stat="identity") +
    labs(y = "Normalized abundance") +
    #scale_y_log10() +
    facet_wrap(~Protein_IDs, nrow=ceiling(n/16),
               ncol=ifelse(n>=16, 16, n),
               scale="free") +
    theme(axis.text.x = element_text(angle=45, vjust=1, hjust = 1))  +
    geom_errorbar(aes(ymin=NormalizedAbundance-sd,
                      ymax=NormalizedAbundance+sd),
                  width=.2,
                  position=position_dodge(.9))+
    ggtitle(paste0("Significant proteins"))
  return(list(plot=p, sig.Prot.nr=n))
}

#rslt.list[[i]][1:3, 1:10]

lapply(names(rslt.list), function(i) {
  mat = rslt.list[[i]]
  design_matrix = design.table.list[[i]]
  plotsigProt = plotProteins(mat, design_matrix)
  n = plotsigProt$sig.Prot.nr
  h=ceiling(n/16)*1.5+1
```

```
  w=ifelse(n>=16, 16*1.5+1.5, n*1.5+1.5)
  pdf(paste0(OUTDIR_DA, "SigDiffProteins_", i, ".pdf"), h=h, w=w)
  print(plotsigProt$plot)
  invisible(dev.off())
 })
```

```
[[1]]
pdf
  2

[[2]]
pdf
  2

[[3]]
pdf
  2
```

6.3 How many proteins are shared: upset plot

```
makeUpset = function(log2FoldChange_thresh, padj_thresh) {
  protein.list = lapply(names(rslt.list), function(i) {
    up = as.data.frame(rslt.list[[i]]) %>%
      filter(logFC>log2FoldChange_thresh&adj.P.Val<padj_thresh)
    down = as.data.frame(rslt.list[[i]]) %>%
      filter(logFC<log2FoldChange_thresh&adj.P.Val<padj_thresh)
    list(up$Protein_IDs, down$Protein_IDs)
  })  %>% setNames(names(rslt.list))

  protein.list_UP = c(protein.list[[1]][1], protein.list[[2]][1], protein.list[[3]][1]) %>
    setNames(c(paste0(names(protein.list)[1], "_UP"),
               paste0(names(protein.list)[2], "_UP"),
               paste0(names(protein.list)[3], "_UP")))
  protein.list_DOWN = c(protein.list[[1]][2], protein.list[[2]][2], protein.list[[3]][2])
    setNames(c(paste0(names(protein.list)[1], "_DOWN"),
               paste0(names(protein.list)[2], "_DOWN"),
               paste0(names(protein.list)[3], "_DOWN")))

  # library(ComplexHeatmap)
  pdf(paste0(OUTDIR_DA, "/Sig.Diff.proteins_upset.pdf"), h=3.5, w=6)
  m = make_comb_mat(c(protein.list_UP, protein.list_DOWN))
  print(UpSet(m,
```

```
        top_annotation = upset_top_annotation(m, add_numbers = TRUE),
        right_annotation = upset_right_annotation(m, add_numbers = TRUE),
        comb_order = rev(order(comb_size(m))),
        set_order = c(paste0(names(protein.list)[1], "_UP"),
                      paste0(names(protein.list)[2], "_UP"),
                      paste0(names(protein.list)[3], "_UP"),
                      paste0(names(protein.list)[1], "_DOWN"),
                      paste0(names(protein.list)[2], "_DOWN"),
                      paste0(names(protein.list)[3], "_DOWN")),
        column_title = paste0("Significant proteins\n(adj.P.Val<", padj_thresh, ")")))

  invisible(dev.off())

  return(list(protein.list_UP=protein.list_UP, protein.list_DOWN=protein.list_DOWN))
}

log2FoldChange_thresh = 0
padj_thresh = 0.2
protein.list = makeUpset(log2FoldChange_thresh, padj_thresh)

intersect(protein.list$protein.list_DOWN$CYTOSOLIC_DOWN, protein.list$protein.list_UP$PELL
#"P42766"
intersect(protein.list$protein.list_UP$CYTOSOLIC_UP, protein.list$protein.list_DOWN$PELLET
#P58549
```

```
[1] "P42766"
[1] "P58549"
```

6.4 How many shared: statistics

```
source("../src/Overlapping_statistics.R")
pdf(paste0(OUTDIR_DA, "/Overlapping_statistics.pdf"), h=3, w=7.6)
list1 = rslt.list[[1]]
list2 = rslt.list[[2]]
name1 = names(rslt.list)[1]
name2 = names(rslt.list)[2]
deplots(list1, list2, name1, name2)

list1 = rslt.list[[1]]
list2 = rslt.list[[3]]
name1 = names(rslt.list)[1]
name2 = names(rslt.list)[3]
```

```r
deplots(list1, list2, name1, name2, Chisq_test=F)

list1 = rslt.list[[2]]
list2 = rslt.list[[3]]
name1 = names(rslt.list)[2]
name2 = names(rslt.list)[3]
deplots(list1, list2, name1, name2, Chisq_test=F)
invisible(dev.off())
```

7 Functional pathway analysis

7.1 Over representation analysis

Because there are a good number of significant proteins (qval<0.2) and there are too many significant pathways detected using GSEA So here I will use ORA

I will use KEGG database because it has a disease database which includes alcoholism

```r
print("Check whether there are duplicated gene names")
lapply(rslt.list, function(r) {
  length(r$Genes) %>% print()
  length(r$Genes[!duplicated(r$Genes)]) %>% print()
})

print("Yes, there is, we need to keep the one with the lowest adj.p.value")
getEntrez = function(rslt.list, sig.level) {
  entrez_list_sig = lapply(rslt.list, function(r) {
    universe =  arrange(r, adj.P.Val) %>% .[!duplicated(.[, "Genes"]), ] %>%
      mutate(entrez = mapIds(org.Hs.eg.db, keys = Genes,
                             column = "ENTREZID", keytype = "SYMBOL",
                             multiVals = "first")) %>%
      filter(!is.na(entrez)) %>% .[, "entrez", drop=T]
    up = filter(arrange(r, adj.P.Val), adj.P.Val<sig.level) %>%
      filter(logFC>0)
    down = filter(arrange(r, adj.P.Val), adj.P.Val<sig.level) %>%
      filter(logFC<0)
    entrez_up = NULL
    if (nrow(up)>0) {
      #Convert gene symbol to Entrez ID
      entrez =  up[!duplicated(up$Genes), ] %>%
        mutate(entrez = mapIds(org.Hs.eg.db, keys = Genes,
                               column = "ENTREZID", keytype = "SYMBOL",
                               multiVals = "first")) %>%
```

```r
        filter(!is.na(entrez))
      entrez_up = entrez[, "entrez", drop=T] %>%
        setNames(., entrez[, "Genes", drop=T])
    }
    entrez_down = NULL
    if (nrow(down)>0) {
      #Convert gene symbol to Entrez ID
      entrez =  down[!duplicated(down$Genes), ] %>%
        mutate(entrez = mapIds(org.Hs.eg.db, keys = Genes,
                               column = "ENTREZID", keytype = "SYMBOL",
                               multiVals = "first")) %>%
        filter(!is.na(entrez))
      entrez_down = entrez[, "entrez", drop=T]  %>%
        setNames(entrez[, "Genes", drop=T])
    }
    return(list(up=entrez_up, down=entrez_down,
                backgrounGeneList=universe))
  })
  return(entrez_list_sig)
}

entrez_list_sig = getEntrez(rslt.list, sig.level=0.2)
OUTDIR_ORA = paste0(outDir, "/PathwayAnalysis/ORA/")
```

```
[1] "Check whether there are duplicated gene names"
[1] 4083
[1] 4063
[1] 4372
[1] 4356
[1] 5183
[1] 5160
$CYTOSOLIC
[1] 4063

$PELLET
[1] 4356

$SYNAPTOSOMAL
[1] 5160

[1] "Yes, there is, we need to keep the one with the lowest adj.p.value"
```

```r
# library(clusterProfiler)
print("KEGG GSEA analysis")
# library(xlsx)
runkk = function(entrez_list_sig) {
  kk.sig = lapply(names(entrez_list_sig), function(i) {
    gl = entrez_list_sig[[i]]
    #saveRDS(gl, paste0(OUTDIR_ORA, "gl.RDS"))
    up = gl$up %>% as.character()
    down = gl$down %>% as.character()
    #universe = gl$backgrounGeneList
    kk_up = NULL
    if (length(up)>0) {
      set.seed(1234)
      kk_up = enrichKEGG(gene = up,
                         organism = 'hsa',
                         pvalueCutoff = 1,
                         qvalueCutoff = 1,
                         #universe = universe,
                         use_internal_data = F)
      print("Convert Entrez ID back to gene symbol")
      source("../src/convertEntrez2Symbol.R")
      kk_up@result$geneID_Symbol =
        convertEntrez2Symbol(kk_up@result$geneID, mapping_list=gl$up) %>%
        as.character()
      file = ifelse(length(entrez_list_sig)==1,
                    paste0(OUTDIR_ORA, "KEGG_ORA_all.xlsx"),
                    paste0(OUTDIR_ORA, "KEGG_ORA.xlsx"))
      if (file.exists(file)) {
        file.remove(file)}
      write.xlsx(as.data.frame(kk_up@result),
                 file,
                 sheetName = paste0(i, "_up"),
                 col.names=TRUE, row.names=FALSE,
                 append=T)
    }

    kk_down = NULL
    if (length(down)>0) {
      set.seed(1234)
      kk_down = enrichKEGG(gene = down,
                           organism = 'hsa',
                           pvalueCutoff = 1,
                           qvalueCutoff = 1,
                           #universe = universe,
```

```
                              use_internal_data = F)
      print("Convert Entrez ID back to gene symbol")
      kk_down@result$geneID_Symbol =
        convertEntrez2Symbol(kk_down@result$geneID, mapping_list=gl$down) %>%
        as.character()
      write.xlsx(as.data.frame(kk_down@result),
                 file,
                 sheetName = paste0(i, "_down"),
                 col.names=TRUE, row.names=FALSE,
                 append=T)
    }
    return(list(up=kk_up, down=kk_down))
  }) %>% setNames(names(entrez_list_sig))
  return(kk.sig)
}

kk.sig = runkk(entrez_list_sig)

saveRDS(kk.sig, paste0(OUTDIR_ORA, "KEGG_ORA_result.RDS"))
# kk.sig = readRDS(paste0(OUTDIR_ORA, "KEGG_ORA_result.RDS"))
```

```
[1] "KEGG GSEA analysis"
[1] "Convert Entrez ID back to gene symbol"
[1] "Convert Entrez ID back to gene symbol"
[1] "Convert Entrez ID back to gene symbol"
[1] "Convert Entrez ID back to gene symbol"
```

7.1.1 Dotplot showing significant KEGG pathways

```
source("../src/makeDotplot.R")
p1 = makeDotplot_ORA_KEGG(rslt=kk.sig, index.list=1:2,
                          qval.thresh=0.1)
pdf(paste0(OUTDIR_ORA, "/KEGG_Pathways_DotPlot_qvalueCutoff0.1.pdf"), h=16.5, w=10)
p1
dev.off()

p1 = makeDotplot_ORA_KEGG(kk.sig, index.list=1:2,
                          qval.thresh=0.05)
pdf(paste0(OUTDIR_ORA, "/KEGG_Pathways_DotPlot_qvalueCutoff0.05.pdf"), h=10, w=10)
p1
dev.off()
```

```
[1] "Combine the ORA result to a long table..."
[1] "Sort the pathways to shared and unique..."
pdf
  2
[1] "Combine the ORA result to a long table..."
[1] "Sort the pathways to shared and unique..."
pdf
  2
```

7.1.2 Check the raw count

```
print("Because some pathways have opposite regualtion directions: up or down, between CYTO
print("Therefore we will give a check using raw data here")
print("For the genes within the ribosome (hsa03010) pathway:")

plotProteins_Raw = function(mat, design_matrix, pathway, compartment="CYTOSOLIC",
                            regulation.direction="down") {
  #saveRDS(list(mat=mat, design_matrix=design_matrix, pathway=pathway, compartment=compart
  #all = readRDS(paste0(OUTDIR_ORA, "all.RDS"))
  pathway.genes =
    strsplit(kk.sig[[compartment]][[regulation.direction]][pathway, "geneID_Symbol"], "/")
      .[[1]]
  #saveRDS(pathway.genes, paste0(OUTDIR_ORA, "pathay.genes.RDS"))
  #pathay.genes = readRDS(paste0(OUTDIR_ORA, "pathay.genes.RDS"))
  pathway.proteins = dataTable.list_SL_IRS_TMM[[compartment]] %>%
    .[.[, "Genes"] %in% pathway.genes, ] %>%
    .[, "Protein_IDs"]
  #saveRDS(pathway.proteins, paste0(OUTDIR_ORA, "pathway.proteins.RDS"))
  #pathway.proteins = readRDS(paste0(OUTDIR_ORA, "pathway.proteins.RDS"))
  #saveRDS(mat, paste0(OUTDIR_ORA, "mat.RDS"))
  #mat = readRDS(paste0(OUTDIR_ORA, "mat.RDS"))
  nCounts = mat %>% as.data.frame() %>%
    gather(-c(Protein_IDs, Gene_names),
           key="sample", value="RawAbundance") %>%
    full_join(design_matrix, by=c("sample")) %>%
    filter(Protein_IDs %in% pathway.proteins)
  #library(plyr)
  #saveRDS(nCounts, paste0(OUTDIR_ORA, "nCounts.RDS"))
  #cCounts = readRDS(paste0(OUTDIR_ORA, "nCounts.RDS"))
  df = ddply(nCounts, .(Batch, Condition, Protein_IDs), summarize,
             mean = round(mean(RawAbundance), 2),
             sd = round(sd(RawAbundance), 2)) #%>%
    #rename(.,c("RawAbundance" = "mean"))
```

```
  df$RawAbundance = df$mean
  #df$RawAbundance = exp(df$RawAbundance)
  df$Protein_IDs = factor(df$Protein_IDs,
                          levels=unique(nCounts$Protein_IDs))
  #saveRDS(df, paste0(OUTDIR_ORA, "df.RDS"))
  #df = readRDS(paste0(OUTDIR_ORA, "df.RDS"))
  n = length(unique(nCounts$Protein_IDs))
  #saveRDS(n, paste0(OUTDIR_ORA, "n.RDS"))
  theme_set(theme_bw())
  p = ggplot(df, aes(fill=Condition, y=RawAbundance, x=Batch)) +
    geom_bar(position="dodge", stat="identity") +
    labs(y = "Raw abundance") +
    #scale_y_log10() +
    facet_wrap(~Protein_IDs, nrow=ceiling(n/16),
               ncol=ifelse(n>=16, 16, n),
               scale="free") +
    theme(axis.text.x = element_text(angle=45, vjust=1, hjust = 1))  +
    geom_errorbar(aes(ymin=RawAbundance-sd,
                      ymax=RawAbundance+sd),
                  width=.2,
                  position=position_dodge(.9))+
    ggtitle(paste0("Significant proteins"))
  return(list(plot=p, sig.Prot.nr=n))
}


dir.create(paste0(OUTDIR_ORA, "/Checking/"),
           showWarnings = F,
           recursive = T)
print("For ribosome pathway (hsa03010)")
pathway = "hsa03010"
compartment = "CYTOSOLIC"
mat = dataTable.list1[[compartment]]
print("Check the data with the raw data table ´Data result_PELLETS (CPJS05122201-02).xlsx´
#head(mat)
print("It seems the same!")
design_matrix = design.table.list[[compartment]]
regulation.direction= "down"
plotPathwayProt = plotProteins_Raw(mat, design_matrix,
                                   pathway,
                                   compartment=compartment,
                                   regulation.direction=regulation.direction)
n = plotPathwayProt$sig.Prot.nr
h=ceiling(n/16)*1.5+1
```

```r
w=ifelse(n>=16, 16*1.5+1.5, n*1.5+1.5)
pdf(paste0(OUTDIR_ORA, "/Checking/",
           pathway,"_",compartment,".pdf"),
    h=h, w=w)
  print(plotPathwayProt$plot)
dev.off()

compartment = "PELLET"
mat = dataTable.list1[[compartment]]
print("Check the data with the raw data table ´Data result_PELLETS (CPJS05122201-02).xlsx´
#head(mat)
print("It seems the same!")
design_matrix = design.table.list[[compartment]]
regulation.direction = "up"
plotPathwayProt = plotProteins_Raw(mat, design_matrix,
                                   pathway,
                                   compartment=compartment,
                                   regulation.direction=regulation.direction)
n = plotPathwayProt$sig.Prot.nr
h=ceiling(n/16)*1.5+1
w=ifelse(n>=16, 16*1.5+1.5, n*1.5+1.5)
pdf(paste0(OUTDIR_ORA, "/Checking/",
           pathway,"_",compartment,".pdf"),
    h=h, w=w)
  print(plotPathwayProt$plot)
dev.off()
print("Again the raw data also show opposite gene expression pattern
      between Cytosolic and Pellet compartments")
print("Note: in the plot, NA: pooled samples/deleted samples")
```

```
[1] "Because some pathways have opposite regualtion directions: up or down, between CYTOSOL
[1] "Therefore we will give a check using raw data here"
[1] "For the genes within the ribosome (hsa03010) pathway:"
[1] "For ribosome pathway (hsa03010)"
[1] "Check the data with the raw data table ´Data result_PELLETS (CPJS05122201-02).xlsx´"
[1] "It seems the same!"
pdf
  2
[1] "Check the data with the raw data table ´Data result_PELLETS (CPJS05122201-02).xlsx´"
[1] "It seems the same!"
pdf
  2
[1] "Again the raw data also show opposite gene expression pattern \n      between Cytosolic
[1] "Note: in the plot, NA: pooled samples/deleted samples"
```

7.1.3 Look into genes associated with pathways of interest

```
print("Check whether there is duplicated gene names")
lapply(rslt.list, function(r) {
  length(r$Genes) %>% print()
  length(r$Genes[!duplicated(r$Genes)]) %>% print()
})
print("Yes, there is, we need to keep the one with the lowest adj.p.value")
entrez_list = lapply(rslt.list, function(r) {
  #Convert gene symbol to Entrez ID
  mat_logFC =  arrange(r, adj.P.Val) %>% .[!duplicated(.[,"Genes"]), ] %>%
    .[, c("Genes", "logFC", "P.Value")]
  entrez = mat_logFC %>%
    #tibble::rownames_to_column(., "Genes") %>%
    mutate(entrez = mapIds(org.Hs.eg.db, keys = Genes,
                           column = "ENTREZID", keytype = "SYMBOL",
                           multiVals = "first")) %>%
    filter(!is.na(entrez)) %>%
    mutate(ranking_statistic = -log10(P.Value)*sign(logFC)) %>%
    arrange(desc(ranking_statistic))
  setNames(entrez$ranking_statistic, entrez$entrez)
})

print("Show the genes in the upregulated pathways for PELLET:")
lapply(kk.sig$PELLET$up@result %>%
         filter(qvalue < 0.1) %>%
         .[, "ID"], function(pathway) {
  node.sum = "sum"
  dir.create(paste0(OUTDIR_ORA, "/Pellet"), showWarnings = F)
  h = pathview(gene.data=entrez_list$PELLET,
                   kegg.dir = paste0(OUTDIR_ORA, "/Pellet/"),
                   expand.node = TRUE, kegg.native=T,
                   pathway.id=pathway, node.sum=node.sum,
                   species="hsa", both.dirs = T, same.layer = F,
                   limit = list(gene=max(abs(entrez_list$PELLET)), cpd=1))
  file.copy(from = paste0("./", pathway, ".pathview.png"),
          to   = paste0(OUTDIR_ORA, "/Pellet/"))
  file.remove(paste0("./", pathway, ".pathview.png"))
  #file.remove(paste0(OUTDIR_ORA, "/Pellet/", pathway, ".png"))
  #file.remove(paste0(OUTDIR_ORA, "/Pellet/", pathway, ".xml"))
})

print("Show the genes in the downregulated pathways for PELLET:")
lapply(kk.sig$PELLET$down@result %>%
```

```
        filter(qvalue < 0.1) %>%
        .[, "ID"], function(pathway) {
  #print(pathway)
  node.sum = "sum"
  dir.create(paste0(OUTDIR_ORA, "/Pellet"), showWarnings = F)
  h = try(pathview(gene.data=entrez_list$PELLET,
                   kegg.dir = paste0(OUTDIR_ORA, "/Pellet/"),
                   expand.node = TRUE, kegg.native=T,
                   pathway.id=pathway, node.sum=node.sum,
                   species="hsa", both.dirs = T, same.layer = F,
                   limit = list(gene=max(abs(entrez_list$PELLET)), cpd=1)),
           TRUE)
  try(file.copy(from = paste0("./", pathway, ".pathview.png"),
          to   = paste0(OUTDIR_ORA, "/Pellet/")), TRUE)
  try(file.remove(paste0("./", pathway, ".pathview.png")), TRUE)
  #file.remove(paste0(OUTDIR_ORA, "/Pellet/", pathway, ".png"))
  #file.remove(paste0(OUTDIR_ORA, "/Pellet/", pathway, ".xml"))
})

print("Show the genes in the upregulated pathways for CYTOSOLIC:")
lapply(kk.sig$CYTOSOLIC$up@result %>%
        filter(qvalue < 0.1) %>%
        .[, "ID"], function(pathway) {
  node.sum = "sum"
  dir.create(paste0(OUTDIR_ORA, "/Cytosolic"), showWarnings = F)
  pathview(gene.data=entrez_list$CYTOSOLIC,
                   kegg.dir = paste0(OUTDIR_ORA, "/Cytosolic/"),
                   expand.node = TRUE, kegg.native=T,
                   pathway.id=pathway, node.sum=node.sum,
                   species="hsa", both.dirs = T, same.layer = F,
                   limit = list(gene=max(abs(entrez_list$PELLET)), cpd=1))
  file.copy(from = paste0("./", pathway, ".pathview.png"),
          to   = paste0(OUTDIR_ORA, "/Cytosolic/"))
  file.remove(paste0("./", pathway, ".pathview.png"))
  #file.remove(paste0(OUTDIR_ORA, "/Pellet/", pathway, ".png"))
  #file.remove(paste0(OUTDIR_ORA, "/Pellet/", pathway, ".xml"))
})

print("Show the genes in the downregulated pathways for CYTOSOLIC:")
lapply(kk.sig$CYTOSOLIC$down@result %>%
        filter(qvalue < 0.1) %>%
        .[, "ID"], function(pathway) {
  node.sum = "sum"
  dir.create(paste0(OUTDIR_ORA, "/Cytosolic"), showWarnings = F)
```

```
  pathview(gene.data=entrez_list$CYTOSOLIC,
                   kegg.dir = paste0(OUTDIR_ORA, "/Cytosolic/"),
                   expand.node = TRUE, kegg.native=T,
                   pathway.id=pathway, node.sum=node.sum,
                   species="hsa", both.dirs = T, same.layer = F,
                   limit = list(gene=max(abs(entrez_list$PELLET)), cpd=1))
  file.copy(from = paste0("./", pathway, ".pathview.png"),
          to   = paste0(OUTDIR_ORA, "/Cytosolic/"))
  file.remove(paste0("./", pathway, ".pathview.png"))
  #file.remove(paste0(OUTDIR_ORA, "/Pellet/", pathway, ".png"))
  #file.remove(paste0(OUTDIR_ORA, "/Pellet/", pathway, ".xml"))
})

print("Color: ranking statstic (Pvalue*sign(logFC)), ")
print("or sum of the rankign statistic if there are multiple genes within a gene node")
print("Which genes are shown: all genes from the dataset that belong to one pathway,
      not only limited to significant genes!")

print("How to find out which genes are within each gene node?")
print("Solution I: bioinformatically: e.g ")
pathway="hsa03010"
clusterProfiler::browseKEGG(kk.sig$CYTOSOLIC$down, pathway)
print("Solution II: open manually through a web browser: e.g.")
print("http://www.kegg.jp/kegg-bin/show_pathway?hsa03010/11224/6139/23521/6141/6143/6154/6
print("Where you have 3 parts:")
print("Part I: http://www.kegg.jp/kegg-bin/show_pathway?")
print("PartII: pathID, i.e. hsa03010 in this case")
print("PartIII: / + item from the geneID column of the KEGG_ORA.xlsx file:")
print("/11224/6139/23521/6141/6143/6154/6165/9349")
```

```
[1] "Check whether there is duplicated gene names"
[1] 4083
[1] 4063
[1] 4372
[1] 4356
[1] 5183
[1] 5160
$CYTOSOLIC
[1] 4063

$PELLET
[1] 4356

$SYNAPTOSOMAL
```

```
[1] 5160

[1] "Yes, there is, we need to keep the one with the lowest adj.p.value"
[1] "Show the genes in the upregulated pathways for PELLET:"
[[1]]
[1] TRUE

[[2]]
[1] TRUE

[[3]]
[1] TRUE

[[4]]
[1] TRUE

[[5]]
[1] TRUE

[[6]]
[1] TRUE

[[7]]
[1] TRUE

[1] "Show the genes in the downregulated pathways for PELLET:"
[[1]]
[1] TRUE

[[2]]
[1] TRUE

[[3]]
[1] TRUE

[[4]]
[1] TRUE

[[5]]
[1] TRUE

[[6]]
[1] TRUE

[[7]]
```

```
[1] TRUE

[[8]]
[1] TRUE

[[9]]
[1] TRUE

[[10]]
[1] TRUE

[[11]]
[1] TRUE

[[12]]
[1] TRUE

[[13]]
[1] TRUE

[[14]]
[1] TRUE

[[15]]
[1] TRUE

[[16]]
[1] TRUE

[[17]]
[1] TRUE

[[18]]
[1] TRUE

[[19]]
[1] TRUE

[[20]]
[1] TRUE

[[21]]
[1] TRUE

[[22]]
```

```
[1] TRUE

[[23]]
[1] TRUE

[[24]]
[1] TRUE

[[25]]
[1] TRUE

[[26]]
[1] TRUE

[[27]]
[1] TRUE

[[28]]
[1] TRUE

[[29]]
[1] TRUE

[[30]]
[1] TRUE

[[31]]
[1] TRUE

[[32]]
[1] TRUE

[[33]]
[1] TRUE

[[34]]
[1] TRUE

[[35]]
[1] TRUE

[[36]]
[1] TRUE

[[37]]
```

```
[1] TRUE

[[38]]
[1] TRUE

[[39]]
[1] TRUE

[[40]]
[1] TRUE

[[41]]
[1] TRUE

[[42]]
[1] TRUE

[[43]]
[1] TRUE

[[44]]
[1] TRUE

[[45]]
[1] TRUE

[[46]]
[1] TRUE

[[47]]
[1] TRUE

[[48]]
[1] TRUE

[[49]]
[1] TRUE

[[50]]
[1] TRUE

[[51]]
[1] TRUE

[[52]]
```

```
[1] TRUE

[[53]]
[1] TRUE

[[54]]
[1] TRUE

[[55]]
[1] TRUE

[[56]]
[1] TRUE

[[57]]
[1] TRUE

[[58]]
[1] TRUE

[[59]]
[1] TRUE

[[60]]
[1] TRUE

[[61]]
[1] TRUE

[[62]]
[1] TRUE

[[63]]
[1] TRUE

[[64]]
[1] TRUE

[1] "Show the genes in the upregulated pathways for CYTOSOLIC:"
[[1]]
[1] TRUE

[[2]]
[1] TRUE
```

```
[[3]]
[1] TRUE

[[4]]
[1] TRUE

[[5]]
[1] TRUE

[[6]]
[1] TRUE

[[7]]
[1] TRUE

[[8]]
[1] TRUE

[[9]]
[1] TRUE

[[10]]
[1] TRUE

[[11]]
[1] TRUE

[[12]]
[1] TRUE

[[13]]
[1] TRUE

[[14]]
[1] TRUE

[[15]]
[1] TRUE

[[16]]
[1] TRUE

[[17]]
[1] TRUE
```

```
[[18]]
[1] TRUE

[[19]]
[1] TRUE

[[20]]
[1] TRUE

[[21]]
[1] TRUE

[[22]]
[1] TRUE

[[23]]
[1] TRUE

[[24]]
[1] TRUE

[[25]]
[1] TRUE

[1] "Show the genes in the downregulated pathways for CYTOSOLIC:"
[[1]]
[1] TRUE

[[2]]
[1] TRUE

[1] "Color: ranking statstic (Pvalue*sign(logFC)), "
[1] "or sum of the rankign statistic if there are multiple genes within a gene node"
[1] "Which genes are shown: all genes from the dataset that belong to one pathway, \n      n
[1] "How to find out which genes are within each gene node?"
[1] "Solution I: bioinformatically: e.g "
[1] "Solution II: open manually through a web browser: e.g."
[1] "http://www.kegg.jp/kegg-bin/show_pathway?hsa03010/11224/6139/23521/6141/6143/6154/6165/
[1] "Where you have 3 parts:"
[1] "Part I: http://www.kegg.jp/kegg-bin/show_pathway?"
[1] "PartII: pathID, i.e. hsa03010 in this case"
[1] "PartIII: / + item from the geneID column of the KEGG_ORA.xlsx file:"
[1] "/11224/6139/23521/6141/6143/6154/6165/9349"
```

> **Note**
>
> In native KEGG view, a gene node may represent multiple genes/proteins with similar or redundant functional role. The number of member genes range from 1 up to several tens. They are intentionally put together as a single node on pathway graphs for better clarity and readability. Therefore, we do not split node and mark each member genes separately by default. But rather we visualize the node-wise data by summarize gene-wise data, users may specify the summarization method using node.sum arguement. Poential options include "sum","mean", "median", "max", "max.abs" and "random". Default node.sum="sum". Here, we used default "sum".

8 Pathway analysis on all three compartments together

```
entrez_list_sig2 = getEntrez(rslt.list, sig.level=0.2)
UP = c()
DOWN = c()
for (i in entrez_list_sig2) {
  UP = c(UP, i$up) %>% .[unique(names(.))]
  DOWN = c(DOWN, i$down) %>% .[unique(names(.))]
}

entrez_list_sig_all = list(all=list(up=UP, down=DOWN))
kk.sig_all = runkk(entrez_list_sig_all)

source("../src/makeDotplot.R")
p1 = makeDotplot_ORA_KEGG(rslt=kk.sig_all, index.list=1,
                        qval.thresh=0.1)
pdf(paste0(OUTDIR_ORA, "/KEGG_Pathways_DotPlot_qvalueCutoff0.1_all.pdf"), h=9, w=8)
p1
dev.off()

p1 = makeDotplot_ORA_KEGG(kk.sig_all, index.list=1,
                        qval.thresh=0.05)
pdf(paste0(OUTDIR_ORA, "/KEGG_Pathways_DotPlot_qvalueCutoff0.05_all.pdf"), h=6, w=8)
p1
dev.off()
```

```
[1] "Convert Entrez ID back to gene symbol"
[1] "Convert Entrez ID back to gene symbol"
[1] "Combine the ORA result to a long table..."
[1] "Sort the pathways to shared and unique..."
pdf
```

```
  2
[1] "Combine the ORA result to a long table..."
[1] "Sort the pathways to shared and unique..."
pdf
  2
```

As you can see that, when combining the significant up and down regulated differential proteins from all three compartments, the pathways got are not as relevant as that of DIABLO, which is probably because this strategy does not account for interactions between different datasets, and is very probably biased toward certain dataset/s.

9 Drug target analysis

```r
dir.create(paste0(OUTDIR_ORA, "/DrugTargets/"), recursive = T, showWarnings = F)
library(enrichR)
listEnrichrSites()

# Select drug related databases:
dbs <- c("LINCS_L1000_Chem_Pert_Consensus_Sigs",
         "Proteomics_Drug_Atlas_2023", "DSigDB",
         "MAGMA_Drugs_and_Diseases", "IDG_Drug_Targets_2022")
# EnrichR analysis for all up/down regulated genes in alcoholism
pdf(paste0(OUTDIR_ORA, "/DrugTargets/EnrichR_top_drug_sets.pdf"))
lapply(c("up", "down"), function(i) {
  enriched <- invisible(enrichr(names(entrez_list_sig_all[[1]][[i]]), dbs))
  for (d in dbs) {
    if (nrow(enriched[[d]])>0) {
      #options(java.parameters = "-Xmx1000m")
      # Plot
      print(plotEnrich(enriched[[d]], showTerms = 20, numChar = 40,
                 y = "Count", orderBy = "P.value",
                 title=paste0(d, "_", i)))
      # Save the result tables
      write.xlsx(enriched[[d]],
                 paste0(OUTDIR_ORA, "/DrugTargets/EnrichR_", i, "_", d,".xlsx"),
                 #sheetName = d,
                 col.names=TRUE, row.names=FALSE,
                 append=F)
    }
  }
})
```

```r
invisible(dev.off())
```

```
Uploading data to Enrichr... Done.
  Querying LINCS_L1000_Chem_Pert_Consensus_Sigs... Done.
  Querying Proteomics_Drug_Atlas_2023... Done.
  Querying DSigDB... Done.
  Querying MAGMA_Drugs_and_Diseases... Done.
  Querying IDG_Drug_Targets_2022... Done.
Parsing results... Done.
Uploading data to Enrichr... Done.
  Querying LINCS_L1000_Chem_Pert_Consensus_Sigs... Done.
  Querying Proteomics_Drug_Atlas_2023... Done.
  Querying DSigDB... Done.
  Querying MAGMA_Drugs_and_Diseases... Done.
  Querying IDG_Drug_Targets_2022... Done.
Parsing results... Done.
[[1]]
NULL

[[2]]
NULL
```

I would rather use the Target analysis results from the DIABLO, rather than from here

```r
sink(paste0(OUTDIR_ORA, "/seesionInfo.txt"))
print(sessionInfo())
sink()
```

```
R version 4.2.2 (2022-10-31)
Platform: x86_64-apple-darwin13.4.0 (64-bit)
Running under: macOS Big Sur ... 10.16

Matrix products: default
BLAS/LAPACK: /Users/ekol-yal/miniconda3/envs/TTN/lib/libopenblasp-r0.3.21.dylib

locale:
[1] en_US.UTF-8/en_US.UTF-8/en_US.UTF-8/C/en_US.UTF-8/en_US.UTF-8

attached base packages:
[1] grid      stats4    stats     graphics  grDevices utils     datasets
[8] methods   base
```

```
other attached packages:
 [1] cowplot_1.1.1               enrichR_3.2
 [3] ggrepel_0.9.3               clusterProfiler_4.6.2
 [5] UniProt.ws_2.38.1           RSQLite_2.3.0
 [7] pathview_1.38.0             org.Hs.eg.db_3.16.0
 [9] AnnotationDbi_1.60.0        GGally_2.1.2
[11] ComplexHeatmap_2.14.0       ggplot2_3.4.4
[13] xlsx_0.6.5                  plyr_1.8.8
[15] tidyr_1.3.0                 dplyr_1.1.4
[17] future.apply_1.10.0         future_1.31.0
[19] edgeR_3.40.2                emmeans_1.8.7
[21] limma_3.54.2                EDASeq_2.32.0
[23] ShortRead_1.56.1            GenomicAlignments_1.34.0
[25] SummarizedExperiment_1.28.0 MatrixGenerics_1.10.0
[27] matrixStats_0.63.0          Rsamtools_2.14.0
[29] GenomicRanges_1.50.2        Biostrings_2.66.0
[31] GenomeInfoDb_1.34.9         XVector_0.38.0
[33] IRanges_2.32.0              S4Vectors_0.36.2
[35] BiocParallel_1.32.5         Biobase_2.58.0
[37] BiocGenerics_0.44.0         purrr_1.0.2
[39] readxl_1.4.2

loaded via a namespace (and not attached):
  [1] estimability_1.4.1     rappdirs_0.3.3         rtracklayer_1.58.0
  [4] R.methodsS3_1.8.2      coda_0.19-4            bit64_4.0.5
  [7] knitr_1.42             aroma.light_3.28.0     multcomp_1.4-23
 [10] DelayedArray_0.24.0    R.utils_2.12.2         data.table_1.14.8
 [13] hwriter_1.3.2.1        KEGGREST_1.38.0        RCurl_1.98-1.10
 [16] doParallel_1.0.17      generics_0.1.3         GenomicFeatures_1.50.4
 [19] TH.data_1.1-2          shadowtext_0.1.2       proxy_0.4-27
 [22] bit_4.0.5              enrichplot_1.18.3      xml2_1.3.3
 [25] viridis_0.6.2          xfun_0.37              hms_1.1.2
 [28] rJava_1.0-6            evaluate_0.20          fansi_1.0.4
 [31] restfulr_0.0.15        progress_1.2.2         dbplyr_2.3.1
 [34] Rgraphviz_2.42.0       igraph_1.4.1           DBI_1.1.3
 [37] reshape_0.8.9          ellipsis_0.3.2         biomaRt_2.54.0
 [40] deldir_1.0-6           vctrs_0.6.4            Cairo_1.6-2
 [43] cachem_1.0.7           withr_2.5.0            ggforce_0.4.1
 [46] HDO.db_0.99.1          treeio_1.22.0          prettyunits_1.1.1
 [49] cluster_2.1.4          DOSE_3.24.2            BiocBaseUtils_1.0.0
 [52] ape_5.7-1              lazyeval_0.2.2         crayon_1.5.2
 [55] ellipse_0.5.0          pkgconfig_2.0.3        labeling_0.4.2
 [58] tweenr_2.0.2           nlme_3.1-162           rlang_1.1.3.9000
 [61] globals_0.16.2         lifecycle_1.0.3        pls_2.8-2
 [64] mt_2.0-1.19            sandwich_3.0-2         downloader_0.4
```

```
 [67] filelock_1.0.2         BiocFileCache_2.6.1    cellranger_1.1.0
 [70] randomForest_4.7-1.1   polyclip_1.10-4        graph_1.76.0
 [73] Matrix_1.6-5           aplot_0.2.2            zoo_1.8-11
 [76] GlobalOptions_0.1.2    png_0.1-8              viridisLite_0.4.1
 [79] rjson_0.2.21           bitops_1.0-7           R.oo_1.25.0
 [82] gson_0.1.0             blob_1.2.3             shape_1.4.6
 [85] stringr_1.5.0          qvalue_2.30.0          parallelly_1.34.0
 [88] jpeg_0.1-10            gridGraphics_0.5-1     scales_1.2.1
 [91] memoise_2.0.1          magrittr_2.0.3         zlibbioc_1.44.0
 [94] compiler_4.2.2         scatterpie_0.1.8       BiocIO_1.8.0
 [97] RColorBrewer_1.1-3     clue_0.3-64            KEGGgraph_1.58.3
[100] cli_3.6.1              listenv_0.9.0          patchwork_1.1.2
[103] MASS_7.3-58.2          mgcv_1.8-42            tidyselect_1.2.0
[106] stringi_1.7.12         yaml_2.3.7             GOSemSim_2.24.0
[109] locfit_1.5-9.7         latticeExtra_0.6-30    fastmatch_1.1-3
[112] rjsoncons_1.0.0        tools_4.2.2            parallel_4.2.2
[115] circlize_0.4.15        rstudioapi_0.15.0      foreach_1.5.2
[118] gridExtra_2.3          farver_2.1.1           ggraph_2.1.0
[121] digest_0.6.31          Rcpp_1.0.10            WriteXLS_6.6.0
[124] httr_1.4.5             colorspace_2.1-0       XML_3.99-0.13
[127] fs_1.6.1               splines_4.2.2          yulab.utils_0.1.0
[130] tidytree_0.4.2         graphlayouts_0.8.4     xlsxjars_0.6.1
[133] ggplotify_0.1.0        xtable_1.8-4           jsonlite_1.8.4
[136] ggtree_3.6.2           tidygraph_1.2.3        ggfun_0.1.3
[139] R6_2.5.1               pillar_1.9.0           htmltools_0.5.4
[142] glue_1.7.0.9000        fastmap_1.1.1          class_7.3-21
[145] codetools_0.2-19       fgsea_1.24.0           mvtnorm_1.1-3
[148] utf8_1.2.3             lattice_0.20-45        tibble_3.2.1
[151] curl_4.3.3             magick_2.7.3           GO.db_3.16.0
[154] interp_1.1-3           survival_3.5-3         rmarkdown_2.25
[157] munsell_0.5.0          e1071_1.7-13           GetoptLong_1.0.5
[160] GenomeInfoDbData_1.2.9 iterators_1.0.14       reshape2_1.4.4
[163] gtable_0.3.1
```

10 Deliverables

11 Directory

11.1 Practical info

11.2 Software

12 Data responsibility

The responsibility for data archiving lies with the PI of the project. We do not offer long-term storage or retrieval of data.

- **NBIS & UPPNEX:** We kindly ask that you remove the files from UPPMAX/UPPNEX. The main storage at UPPNEX is optimized for high-speed and parallel access, which makes it expensive and not the right place for longer time archiving. Please consider others by not taking up the expensive space. Please note that UPPMAX is a resource separate from the Bioinformatics Platform, administered by the Swedish National Infrastructure for Computing (SNIC) and SNIC-specifc project rules apply to all projects hosted at UPPMAX.
- **Sensitive data:** Please note that special considerations may apply to the human-derived legally considered sensitive personal data. These should be handled according to specific laws and regulations as outlined e.g. here.
- **Long-term backup:** We recommend asking your local IT for support with long-term data archiving. The Data Office at SciLifeLab may be of help to discuss other options.

13 Acknowledgments

If you are presenting the results in a paper, at a workshop or conference, we kindly ask you to acknowledge us.

- NBIS staff are encouraged to be co-authors when this is merited in accordance to the ethical recommendations for authorship, e.g. ICMJE recommendations. If applicable, please include **Name, Surname, National Bioinformatics Infrastructure Sweden, Science for Life Laboratory, Further Affliations**, as co-author. In other cases, NBIS would be grateful if support by us is acknowledged in publications according to this example:

  "Support by NBIS (National Bioinformatics Infrastructure Sweden) is gratefully acknowledged."

- **UPPMAX** kindly asks you to acknowledge UPPMAX and NAISS.

- **NGI:** For publications based on data from NGI Sweden, NGI and SciLifeLab should be acknowledged. Instructions are included in the NGI reports as described here.